EXPERIMENT NO – 01

Given a 4-variable logic expression, simplify it using appropriate technique and simulate the same using basic gates.

Aim: Given a 4-variable logic expression, simplify it using appropriate technique and simulate the same using basic gates.

Objectives:

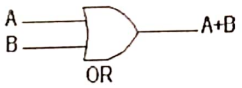
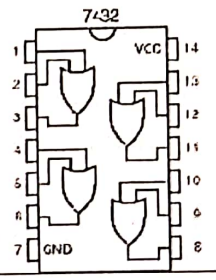
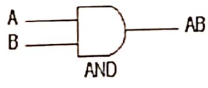
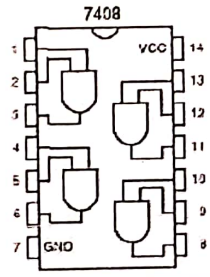
1. Simplify 4-variable logic expression for efficient design.
2. Simulate using basic gates for practical verification and optimization.

Theory:

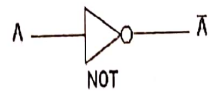
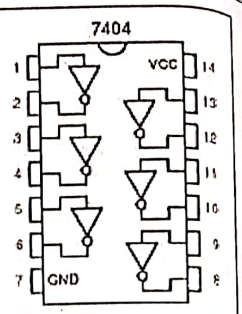
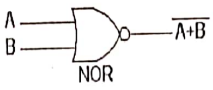
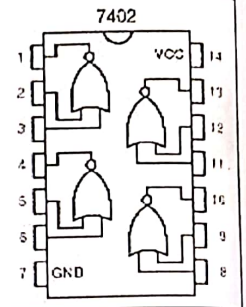
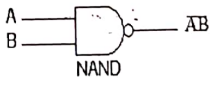
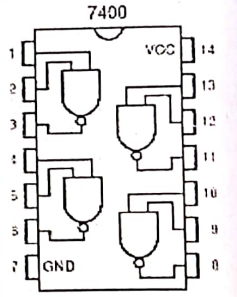
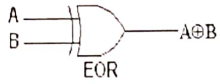
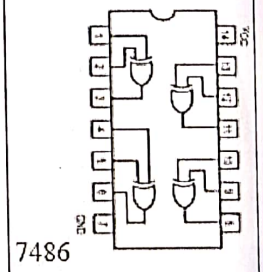
SUM OF PRODUCT:

The short form of the sum of the product is SOP, and it is one kind of **Boolean algebra** expression. In this, the different product inputs are being added together. The product of inputs is Boolean logical AND whereas the sum or addition is Boolean logical OR. Before going to understand the concept of the sum of products

Expression of SOP F=AB+C(D+E)

Gate	Description	Truth Table	Logic Symbol	Pin Diagram
OR	The output is active high if any one of the input is in active high state, Mathematically, Q = A+B	A B Output 0 0 0 0 1 1 1 0 1 1 1 1		
AND	The output is active high only if both the inputs are in active high state, Mathematically,	A B Output 0 0 0 0 1 0 1 0 0 1 1 1		

Scanned with OKEN Scanner

NOT	In this gate the output is opposite to the input state, Mathematically, Q = A	A 0 1	Output 1 0		
NOR	The output is active high only if both the inputs are in active low state, Mathematically, Q = (A+B)'	A B 0 0 0 1 1 0 1 1	Output 1 0 0 0		
NAND	The output is active high only if any one of the input is in active low state, Mathematically, Q = (A.B)'	A B 0 0 0 1 1 0 1 1	Output Q 1 1 1 0		
XOR	The output is active high only if any one of the input is in active high state, Mathematically, Q = A.B' + B.A'	A B 0 0 0 1 1 0 1 1	Output 0 1 1 0		

Equipment / components required:

- Computer with Modelsim
- HP Computer i4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Softwares: Modelsim - 5.7c

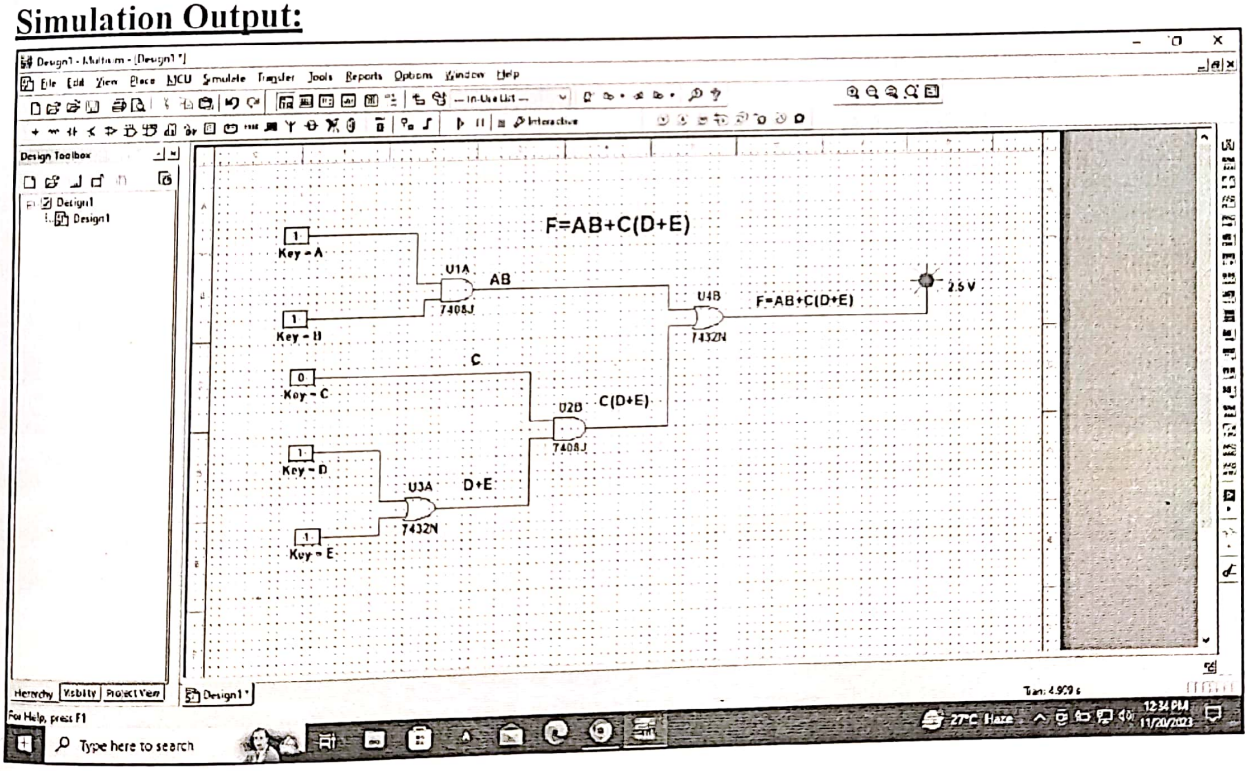
Procedure:

1. Open Multisim software on your computer.
2. Click on "File" and select "New" to create a new schematic.
3. Drag and drop components from the component toolbar onto the workspace.
4. Connect the components using wires.Specify values and properties for components.

Scanned with OKEN Scanner

5. Add a power supply to the circuit. Connect it appropriately (Vcc and GND).
6. Double-click on components to open their properties.
7. Set parameters, such as resistor values, capacitor values, etc.
8. Click on "Simulate" in the toolbar.
9. Set simulation parameters like start time, stop time, and time step.
10. Add instruments to measure and observe circuit behavior.
11. Click on the "Run" button (play icon) to start the simulation.
12. Observe and analyze waveforms and measurements from the instruments. Check for correct circuit behavior.
13. If unexpected results occur, review the circuit, component values, and connections. Make necessary adjustments and rerun the simulation.
14. Save your schematic and simulation results.
15. Document the circuit design, simulation parameters, and observed behavior.
16. Export simulation results or circuit diagrams for sharing or further analysis.

Simulation Output:



Results & conclusions:

The Simulate for Given a 4-variable logic expression, simplify it using appropriate technique using basic gates are verified.

Possible viva questions:

1. What is Multisim?

Scanned with OKEN Scanner

Answer: Multisim is a simulation software developed by National Instruments that allows users to design, simulate, and analyze electronic circuits. It provides a virtual environment for testing circuits before physical implementation

2. **How does Multisim aid in circuit design?**

   Answer: Multisim allows users to create and simulate electronic circuits virtually, enabling them to test the functionality and performance of the circuits without the need for physical components. This aids in the design and validation process.

3. **What types of components can you simulate in Multisim?**

   Answer: Multisim supports a wide range of electronic components, including resistors, capacitors, inductors, transistors, op-amps, microcontrollers, and more. Users can simulate both analog and digital components in their circuits.

4. **What is the purpose of simplifying a logic expression?**

   Answer: Simplifying a logic expression reduces the number of gates needed, leading to more efficient and faster circuits.

5. **How many cells does a 4-variable K-map have?**

Answer: A 4-variable K-map has 16 cells (2^4).

6. **Explain the steps involved in simplifying a Boolean expression using K-maps.**

   Answer: Identify minterms, group adjacent 1s in the K-map, find the minimal sum-of-products (SOP) expression, and then apply any additional simplifications.

7. **Given a 4-variable expression ABCD + A'C'D + A'BD', simplify it using a K-map.**

   Answer: The simplified expression is AD' + A'C'.

8. **Given the expression (A + B)(C + D), simplify it using Boolean algebra.**

   Answer: AC + AD + BC + BD

9. **What is the significance of don't care conditions in logic simplification?**

Answer: Don't care conditions represent input combinations for which the output value is not critical. Utilizing don't care conditions allows for further optimization during simplification.

Scanned with OKEN Scanner

EXPERIMENT NO – 02

Design a 4 bit full adder and subtractor and simulate the same using basic gates.

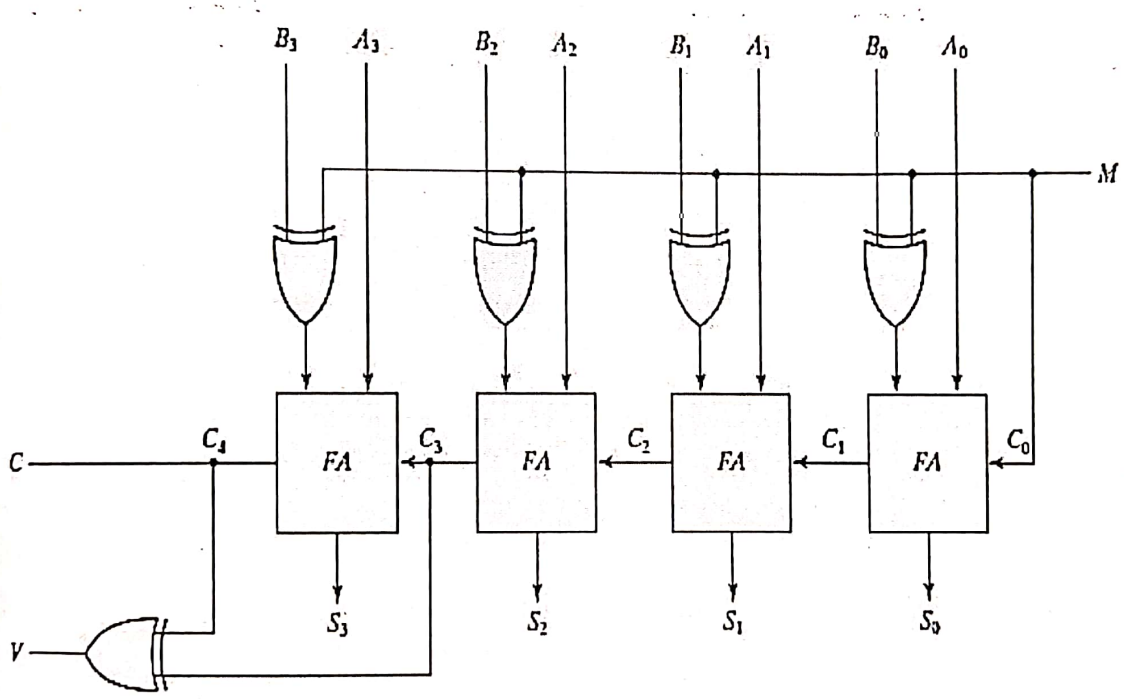
Aim: To design a 4 bit full adder and subtractor and simulate the same using basic gates.

Objectives:

1. Design a 4-bit full adder and subtractor circuit in Multisim using basic gates.
2. Simulate the circuit to analyze and verify correct functionality, considering various input scenarios.

Theory:

The addition and subtraction operations can be combined into one circuit with one common binary adder by including an exclusive-OR gate with each full-adder.



The mode input M controls the operation as the following:

If M=0 → the circuit is an adder

M=1 → the circuit becomes a subtractor

Each exclusive-OR gate receives input M and one of the inputs of B.

- When M = 0, we have B⊕ 0 = B. The full adders receive the value of B, the input carry is $C_0 = 0$, and the circuit performs A + B.

Scanned with OKEN Scanner

- When M = 1, we have B ⊕ 1 = *B*and C_0 = 1. The B inputs are all complemented and a 1 is added through the input carry. The circuit performs the operation A + (the 2's complement of B)= A –B
- The exclusive-OR with output V is for detecting an overflow.

The binary adder–subtractor circuit with outputs C and V is shown in Fig. 4.13. If the two binary numbers are considered to be unsigned, then the C bit detects a carry after addition or a borrow after subtraction. If the numbers are considered to be signed, then the V bit detects an overflow. If V = 0 after an addition or subtraction, then no overflow occurred and the n -bit result is correct. If V = 1, then the result of the operation contains n + 1 bits, but only the rightmost n bits of the number fit in the space available, so an overflow has occurred. The 1n + 12 th bit is the actual sign and has been shifted out of position.

Equipment / components required:

- Computer with Modelsim
- HP Computer i4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Softwares: Modelsim - 5.7c

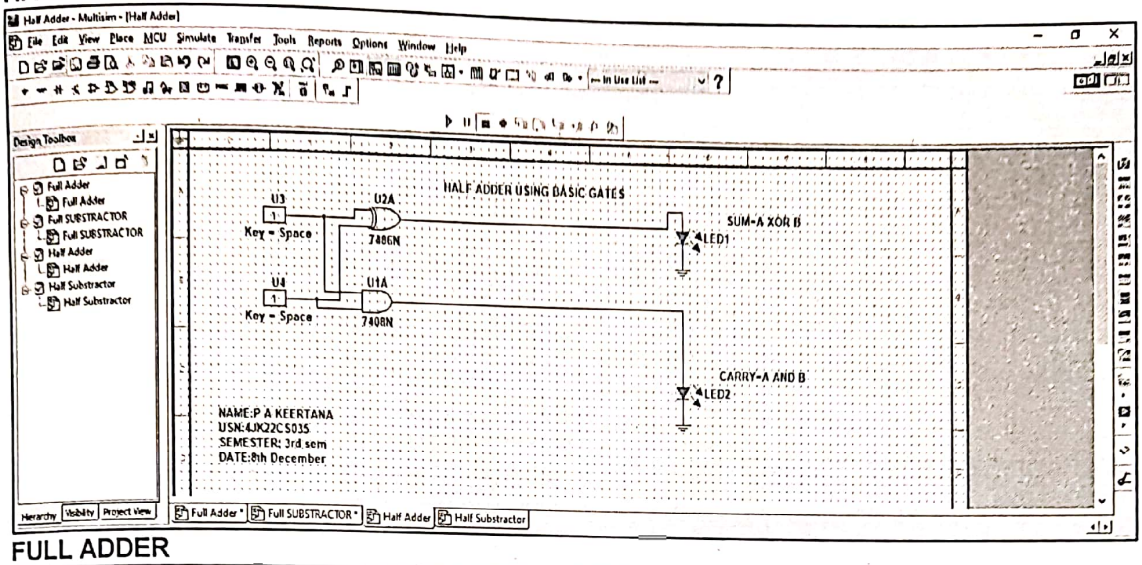
Procedure / activity:

1. Open Multisim software on your computer.
2. Click on "File" and select "New" to create a new schematic.
3. Drag and drop components from the component toolbar onto the workspace.
4. Connect the components using wires.Specify values and properties for components.
5. Add a power supply to the circuit. Connect it appropriately (Vcc and GND).
6. Double-click on components to open their properties.
7. Set parameters, such as resistor values, capacitor values, etc.
8. Click on "Simulate" in the toolbar.
9. Set simulation parameters like start time, stop time, and time step.
10. Add instruments to measure and observe circuit behavior.
11. Click on the "Run" button (play icon) to start the simulation.
12. Observe and analyze waveforms and measurements from the instruments. Check for correct circuit behavior.
13. If unexpected results occur, review the circuit, component values, and connections. Make necessary adjustments and rerun the simulation.
14. Save your schematic and simulation results.
15. Document the circuit design, simulation parameters, and observed behavior.
16. Export simulation results or circuit diagrams for sharing or further analysis.

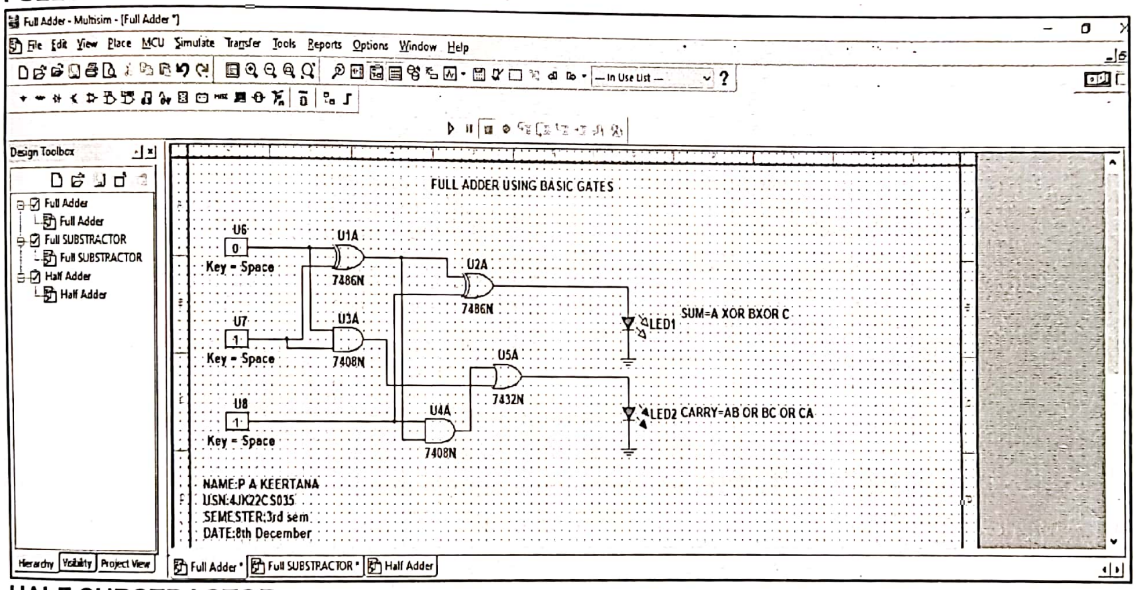
Scanned with OKEN Scanner

Simulation Output:

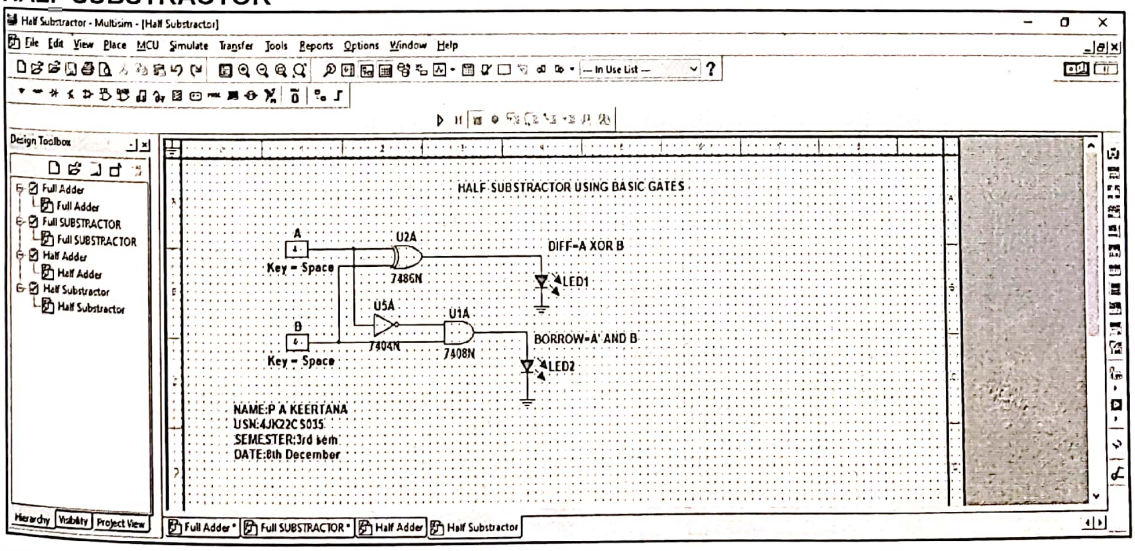
HALF ADDER



FULL ADDER

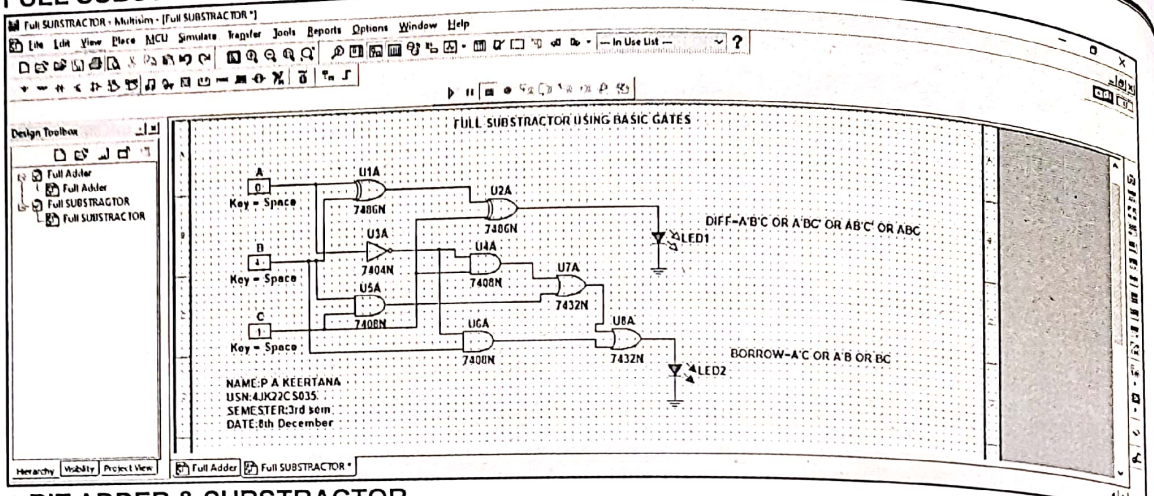


HALF SUBSTRACTOR

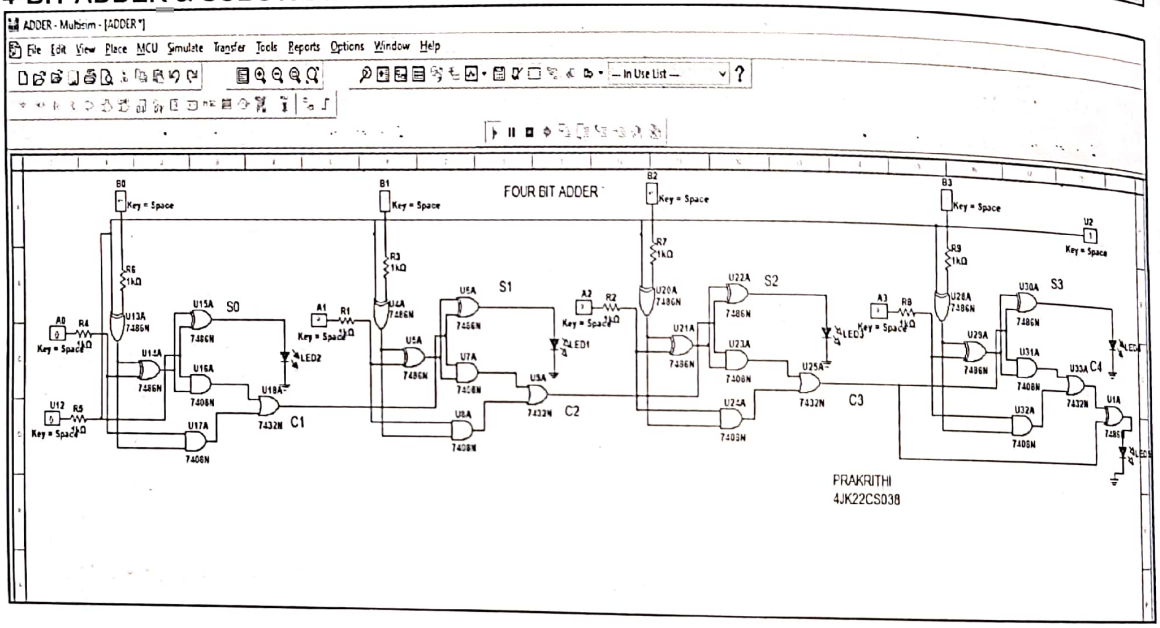


Scanned with OKEN Scanner

FULL SUBSTRACTOR



4-BIT ADDER & SUBSTRACTOR



Results & conclusions: Simulated 4-bit full adder and subtractor in Multisim, verified correct outputs, efficient gate use, and stable performance. Achieved accurate, optimized, and versatile circuit design.

Possible viva questions:

1. **What is the primary function of a 4-bit full adder?**
   A 4-bit full adder performs addition on two 4-bit binary numbers, considering inputs and generating a sum along with a carry-out.

2. **Can a 4-bit full adder handle subtraction as well?**
   Yes, a 4-bit full adder can be used for subtraction by incorporating additional logic to manage borrow inputs and adjustments.

3. **How many control inputs are there in a 4-bit full subtractor for each XOR gate used?**
   2

Scanned with OKEN Scanner

4. **What is the purpose of the AND gates in a full subtractor?**

   A) Generate the sum output
   B) Generate the borrow-out
   C) Perform bitwise AND operation
   D) Implement overflow detection

   Answer: C) Perform bitwise AND operation

5. **List the basic gates required to design a 4-bit full adder.**

   Answer: Basic gates include AND gates, XOR gates, and OR gates.

6. **In a 4-bit full subtractor, what is the role of the borrow-out?**

   A) It indicates a borrow to the next stage.
   B) It determines the most significant bit of the difference.
   C) It controls the XOR gates.
   D) It is not used in subtraction.

   Answer: A) It indicates a borrow to the next stage.

7. **What is the purpose of the AND gates in a full subtractor?**

A) Generate the sum output
B) Generate the borrow-out
C) Perform bitwise AND operation
D) Implement overflow detection

Answer: C) Perform bitwise AND operation

8. **How would you simulate the 4-bit full adder and subtractor using basic gates in a software tool like Multisim or similar?**

   Answer: Create a schematic in the simulation tool, instantiate the required basic gates (AND, XOR, OR), and interconnect them according to the design of the 4-bit full adder and subtractor. Apply appropriate inputs and observe the outputs.

9. **How many XOR gates are required for the sum output in a 4-bit full adder?**

   Answer: Four XOR gates are required for generating the sum outputs in a 4-bit full adder.

10. **Can you identify any challenges or considerations when designing a 4-bit full subtractor?**

    Answer: One consideration is managing borrow inputs for each bit subtraction, and careful handling of borrow propagation to ensure accurate subtraction results.

Scanned with OKEN Scanner

EXPERIMENT NO – 03

Design Verilog HDL to implement simple circuits using structural, Data flow and Behavioural model.

Aim: To design Verilog HDL to implement simple circuits using structural, Data flow and Behavioural model.

Objectives:

1. Implement Verilog for simple circuits using structural, data flow, and behavioral models.
2. Verify designs through simulation for functionality and performance assessment.

Theory:

As mentioned previously, the module is the basic building block for modeling hardware with the Verilog HDL. The logic of a module can be described in anyone (or a combination) of the following modeling styles:

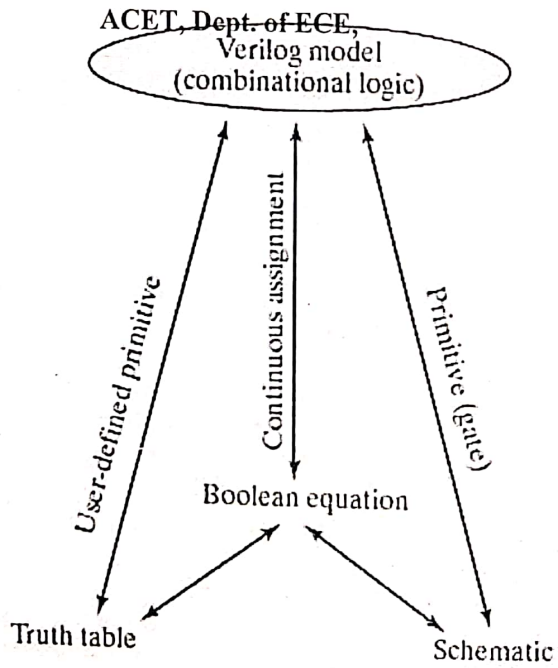


FIGURE 3.1: Relationship of Verilog constructs to truth tables, Boolean equations, and schematics

- **Gate-level modeling** using instantiations of predefined and user-defined primitive gates.
  - Gate-level (structural) modeling describes a circuit by specifying its gates and how they are connected with each other.

Scanned with OKEN Scanner

- **Dataflow modeling** using continuous assignment statements with the keyword assign.
  - Dataflow modeling is used mostly for describing the Boolean equations of combinational logic.
- **Behavioral modeling** using procedural assignment statements with the keyword always.
  - behavioral modeling that is used to describe combinational and sequential circuits at a higher level of abstraction.

Combinational logic can be designed with truth tables, Boolean equations, and schematics; Verilog has a construct corresponding to each of these "classical" approaches to design: user-defined primitives, continuous assignments, and primitives, as shown in Fig. 3.1. There is one other modeling style, called switch-level modeling. It is sometimes used in the simulation of MOS transistor circuit models, but not in logic synthesis. We will not consider switch-level modeling.

Equipment / components required:

- Computer with Modelsim SoftwareSpecifications:
- HP Computer P4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Softwares: Modelsim - 5.7c, Xilinx - 14.1i.

Procedure / activity: ACET, Dept. of ECE,

1. Open a Xilinx IDE, close old projects.
2. Create a new project through the project navigator icon.
3. Add VHDL/Verilog new source to the project depending on the user requirement.
4. Select the Verilog HDL mode.
5. Assign the inputs and outputs for the system to design.
6. And write the code as given below for the functionality.
7. Synthesize the code and correct the syntax errors if any.
8. Isim Simulator and Behavioral check syntax the Simulation behavioral model.
9. Observe the output timing waveform and verify it with the truth table.

Verilog code:

Structural-	Data Flow-	Behavioural Model-

module logicg(a, b,y0,y1,y2,y3,y4,y5,y6);
input a,b;
output y0,y1,y2,y3,y4,y5,y6;
not g1(y0,a);
and g2(y1,a,b);
or g3(y2,a,b);
nand g4(y3,a,b);
nor g5(y4,a,b);
xor g6(y5,a,b);
xnor g7(y6,a,b);
endmodule
	module ldataflow(a, b,y0,y1,y2,y3,y4,y5,y6);
	input a,b;
	output y0,y1,y2,y3,y4,y5,y6;
	assign y0=!a;
	assign y1=a&b;
	assign y2=a|b;
	assign y3=!(a&b);
	assign y4=!(a|b);
	assign y5=a^b;
	assign y6=!(a^b);
	endmodule
		module logicgB(a, b,y0,y1,y2,y3,y4,y5,y6);
		input a,b;
		output y0,y1,y2,y3,y4,y5,y6;
		reg y0;
		reg y1;
		reg y2;
		reg y3;
		reg y4;
		reg y5;
		reg y6;
		always @(a,b)

Scanned with OKEN Scanner

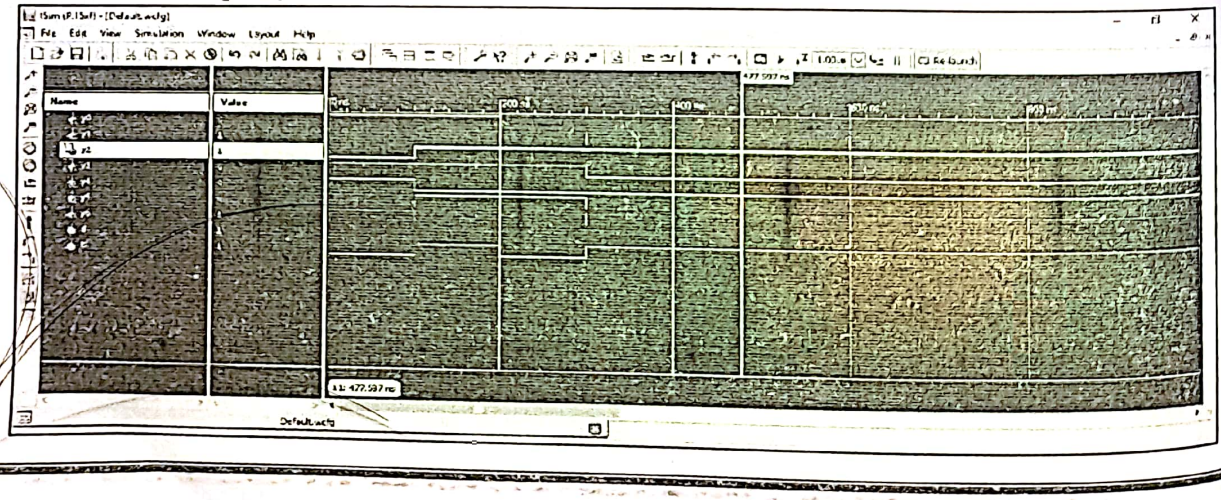
		begin
		y0=!a;
		y1=a&b;
		y2=a|b;
		y3=!(a&b);
		y4=!(a|b);
		y5=a^b;
		y6=!(a^b);
		end
		endmodule

Verilog Testbench code:

Structural-	Data Flow-	Behavioural Model-

module logicg_tb;
reg a;
reg b;
wire y0;
wire y1;
wire y2;
wire y3;
wire y4;
wire y5;
wire y6;
logicg uut (.a(a), .b(b),
.y0(y0), .y1(y1),
.y2(y2), .y3(y3),
.y4(y4), .y5(y5),
.y6(y6));
initial begin
a = 0; b = 0;#100;
a = 0; b = 1;#100;
a = 1; b = 0;#100;
a = 1; b = 1;#100;
end
endmodule
	module ldataflow_tb;
	reg a;
	reg b;
	wire y0;
	wire y1;
	wire y2;
	wire y3;
	wire y4;
	wire y5;
	wire y6;
	ldataflow uut (.a(a), .b(b),
	.y0(y0), .y1(y1),
	.y2(y2), .y3(y3),
	.y4(y4), .y5(y5),
	.y6(y6));
	initial begin
	a = 0; b = 0;#100;
	a = 0; b = 1;#100;
	a = 1; b = 0;#100;
	a = 1; b = 1;#100;
	end
	endmodule
		module logicgB_tb;
		reg a;
		reg b;
		wire y0;
		wire y1;
		wire y2;
		wire y3;
		wire y4;
		wire y5;
		wire y6;
		logicgB uut (.a(a), .b(b),
		.y0(y0), .y1(y1),
		.y2(y2), .y3(y3), .y4(y4),
		.y5(y5), .y6(y6));
		initial begin
		a = 0; b = 0; #100;
		a = 0; b = 1; #100;
		a = 1; b = 0; #100;
		a = 1; b = 1; #100;
		end
		endmodule

Simulation Output:



Scanned with OKEN Scanner

Results & conclusions: The Verilog code for simple circuits using structural, Data flow and Behavioral model are verified.

Possible viva questions:

1. **What is Verilog HDL?**
   Verilog Hardware Description Language (HDL) is a language used for modeling electronic systems at the behavioral and structural levels, commonly employed in digital design and simulation.
2. **Explain the structural modeling in Verilog.**
   Structural modeling involves building circuits using predefined modules. Instances of modules are interconnected to create a complete design.
3. **Explain the concept of continuous assignment in data flow modeling.**
   Continuous assignments use the assign keyword to express the relationship between signals. These assignments operate continuously, updating the output whenever input signals change.
4. **In behavioral modeling, what does the always block signify?**
   The always block in behavioral modeling defines a set of conditions and statements that are executed whenever the conditions specified in the sensitivity list change.
5. **Can Verilog be used for both simulation and synthesis?**
   Yes, Verilog is versatile and can be used for both simulation, to verify designs, and synthesis, to generate hardware based on the design description.
6. **How are signals represented in behavioral modeling in Verilog?**
   In behavioral modeling, signals are represented using variables and expressions. Changes in variables trigger the execution of blocks, simulating the algorithmic behavior.
7. **How is time accounted for in behavioral modeling in Verilog?**
   Behavioral modeling can include delays using constructs like # to represent time delays between statements. This allows simulation to account for time-dependent behavior.
8. **How does structural modeling differ from behavioral modeling in Verilog?**
   Structural modeling focuses on the interconnection of modules, representing the physical structure of the circuit. Behavioral modeling emphasizes the algorithmic functionality, abstracting the physical implementation details.
9. **In which scenarios would you prefer to use structural modeling in Verilog?**
   Structural modeling is beneficial when designing circuits with well-defined modules and known physical implementations, making it suitable for hierarchical designs.
10. **When is behavioral modeling more appropriate in Verilog?**
    Behavioral modeling is preferred for abstractly describing the functionality of a circuit without concern for the specific physical structure. It is useful for algorithmic or higher-level design descriptions.

Scanned with OKEN Scanner

EXPERIMENT NO – 04

Design Verilog HDL to implement Binary Adder-Subtractor – Half and Full Adder, Half and Full Subtractor.

Aim: To Design Verilog HDL to implement Binary Adder-Subtractor – Half and Full Adder, Half and Full Subtractor.

Objectives:

1. Create Verilog HDL for Binary Adder-Subtractor, including Half and Full Adder, Half and Full Subtractor.
2. Validate functionality through simulation for accurate arithmetic operations in digital systems.

Theory:

Adder is a combinational digital circuit that is used for adding two numbers. A typical adder circuit produces a sum bit (denoted by S) and a carry bit (denoted by C) as the output. Adder circuits are of two types: Half adder and Full adder.

Half-Adder: A combinational logic circuit that performs the addition of two data bits, A and B, is called a half-adder. Addition will result in two output bits; one of which is the sum bit, S, and the other is the carry bit, C.

Full-Adder: The half-adder does not take the carry bit from its previous stage into account. This carry bit from its previous stage is called carry-in bit. A combinational logic circuit that adds two data bits, A and B, and a carry-in bit Cin, is called a full-adder.

Subtractor is a combinational digital circuit that is used for subtracting two numbers. A typical subtractor circuit produces a diff bit (denoted by D) and a borrow bit (denoted by B)as the output. Subtractor circuits are of two types: Half subtractor and Full subtractor.

Half Subtractor: A combinational logic circuit that performs the subtraction of two data bits, A and B, is called a half-adder. Subtraction will result in two output bits; one of which is the diff bit, D, and the other is the borrow bit, B.

Full Subtractor: A combinational logic circuit that subtractor two data bits, A and B and a borrow in bit, is called full subtractor.

Scanned with OKEN Scanner

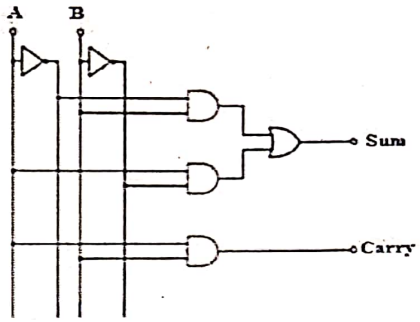
Using Basic Gates

Half Adder

Half Adder

Input		output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

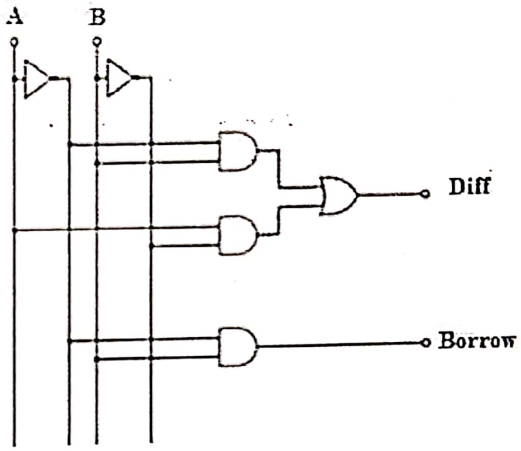
Sum= $\bar{A}B+A\bar{B}$
Carry=AB



Half Subtractor

Input		output	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Difference= $\bar{A}B+A\bar{B}$
Borrow= $\bar{A}B$

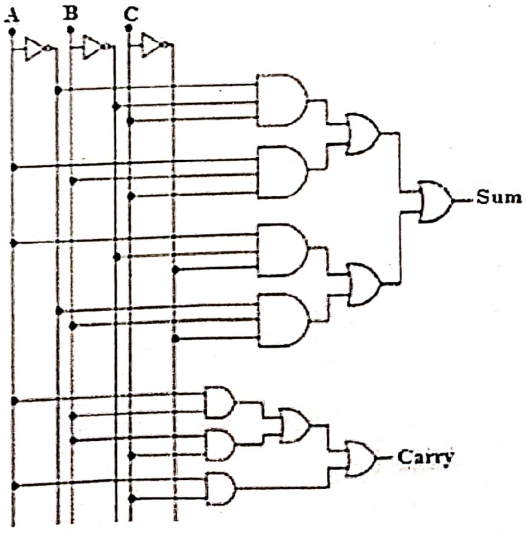


Full Adder

Full adder

Input			output	
A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sum = $\bar{a}\bar{b}c+a\bar{b}\bar{c}+\bar{a}b\bar{c}+abc$
Carry = $a\cdot b+a\cdot c+b\cdot c$



Scanned with OKEN Scanner

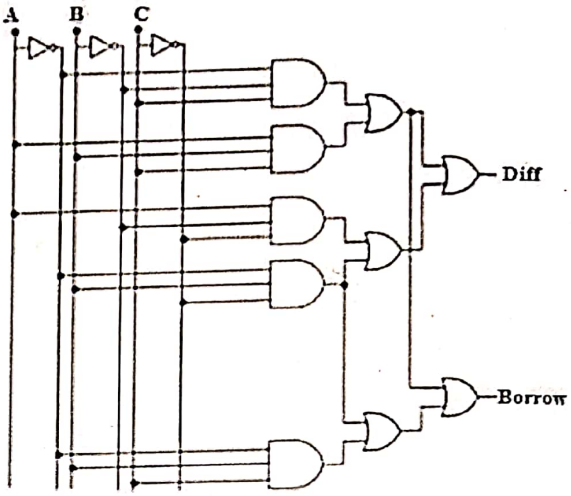
Full Subtractor

Full Subtractor

Input			output	
A	B	C	Difference	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

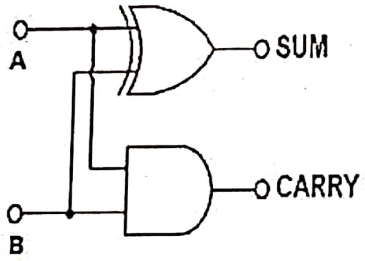
Difference = $\bar{a}\,\bar{b}\,c + \bar{a}\,b\,\bar{c} + a\,\bar{b}\,\bar{c} + a\,b\,c$

Borrow = $\bar{a}\,c + \bar{a}\,b + b\,c$



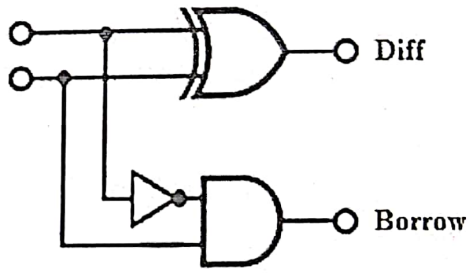
Using EX-OR Gates

Half Adder



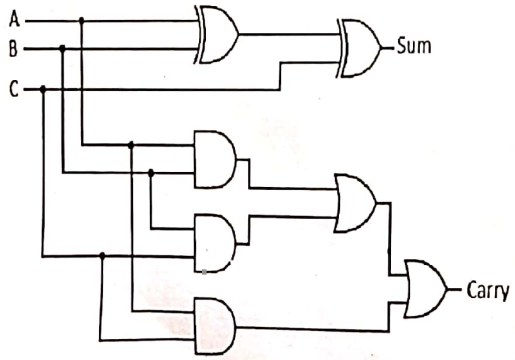
Sum= A⊕B
Carry= AB

Half Subtractor



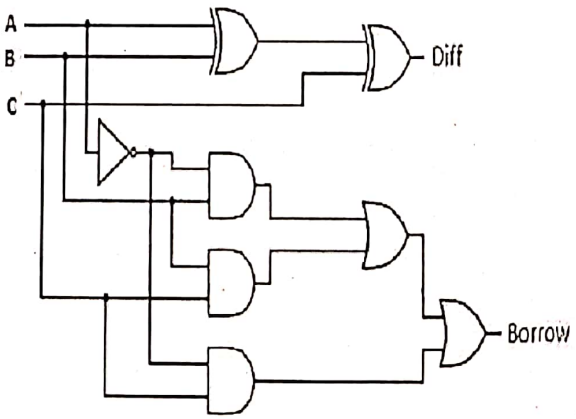
Diff= A⊕B
Borrow=A'B

Full Adder



Sum=A⊕B⊕C
Carry=AB+BC+AC

Full Subtractor



Diff= A⊕B⊕C
Borrow=A'B+BC+A'C

Scanned with OKEN Scanner

Equipment / components required:

- Computer with Modelsim SoftwareSpecifications:
- HP Computer P4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Softwares: Modelsim - 5.7c, Xilinx - 14.1i.

Procedure / activity:

1. Open a Xilinx IDE, close old projects.
2. Create a new project through the project navigator icon.
3. Add VHDL/Verilog new source to the project depending on the user requirement.
4. Select the Verilog HDL mode.
5. Assign the inputs and outputs for the system to design.
6. And write the code as given below for the functionality.
7. Synthesize the code and correct the syntax errors if any.
8. Isim Simulator and Behavioral check syntax the Simulation behavioral model.
9. Observe the output timing waveform and verify it with the truth table.

Verilog code:

Half Adder-	Half Substractor-

module halfadder(a,b,sum,carry);
input a,b;
output sum,carry;
assign sum=a^b;
assign carry=a&b;
endmodule
	module HS(a,b,diff,borrow);
	input a,b;
	output diff,borrow;
	assign diff=a^b;
	assign borrow=!(a)&b;
	endmodule

Full Adder-	**Full Substractor-**

module FA(a,b,c,sum,carry);
input a,b,c;
output sum,carry;
assign sum=a^b^c;
assign carry=(a&b)||(b&c)||(c&a);
endmodule
	module FS(a,b,c,diff,borrow);
	input a,b,c;
	output diff,borrow;
	assign diff=a^b^c;
	assign borrow=((!a)&b)|(b&c)|((!a)&c);
	endmodule

Verilog Testbench code:

Half Adder-	Half Substractor-
module halfadder_tb; reg a; reg b; wire sum; wire carry; halfadder uut (.a(a), .b(b),	module HS_tb; reg a; reg b; wire diff; wire borrow; HS uut (.a(a), .b(b),

Scanned with OKEN Scanner

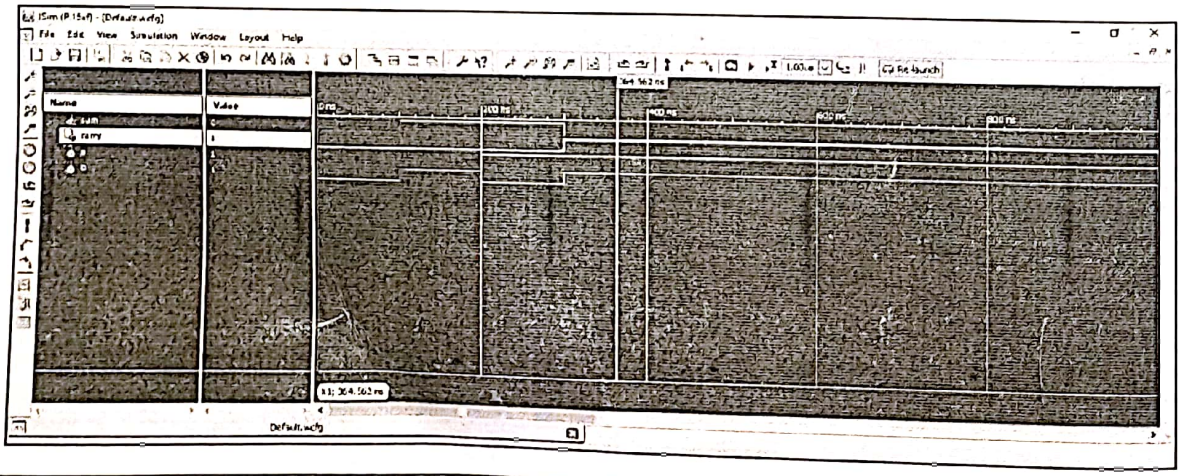
.sum(sum),
.carry(carry));
initial begin
a = 0; b = 0; #100;
a = 0; b = 1; #100;
a = 1; b = 0; #100;
a = 1; b = 1; #100;
end
endmodule
	.diff(diff),
	.borrow(borrow));
	initial begin
	a = 0; b = 0; #100;
	a = 0; b = 1; #100;
	a = 1; b = 0; #100;
	a = 1; b = 1; #100;
	end
	endmodule

Full Adder-	**Full Substractor-**

module FA_tb;
reg a;
reg b;
reg c;
wire sum;
wire carry;
FA uut (
.a(a),
.b(b),
.c(c),
.sum(sum),
.carry(carry));
initial begin
a = 0;b = 0;c = 0;#100;
a = 0;b = 0;c = 1;#100;
a = 0;b = 1;c = 0;#100;
a = 0;b = 1;c = 1;#100;
a = 1;b = 0;c =0;#100;
a = 1;b = 0;c = 1;#100;
a =1;b = 1;c = 0;#100;
a = 1;b = 1;c = 1;#100;
end
endmodule
	module FS_tb;
	reg a;
	reg b;
	reg c;
	wire diff;
	wire borrow;
	FS uut (
	.a(a),
	.b(b),
	.c(c),
	.diff(diff),
	.borrow(borrow));
	initial begin
	a = 0;b = 0;c = 0;#100;
	a = 0;b = 0;c = 1;#100;
	a = 0;b = 1;c = 0;#100;
	a = 0;b = 1;c = 1;#100;
	a = 1;b = 0;c = 0;#100;
	a = 1;b = 0;c = 1;#100;
	a = 1;b = 1;c = 0;#100;
	a = 1;b = 1;c = 1;#100;
	end
	endmodule

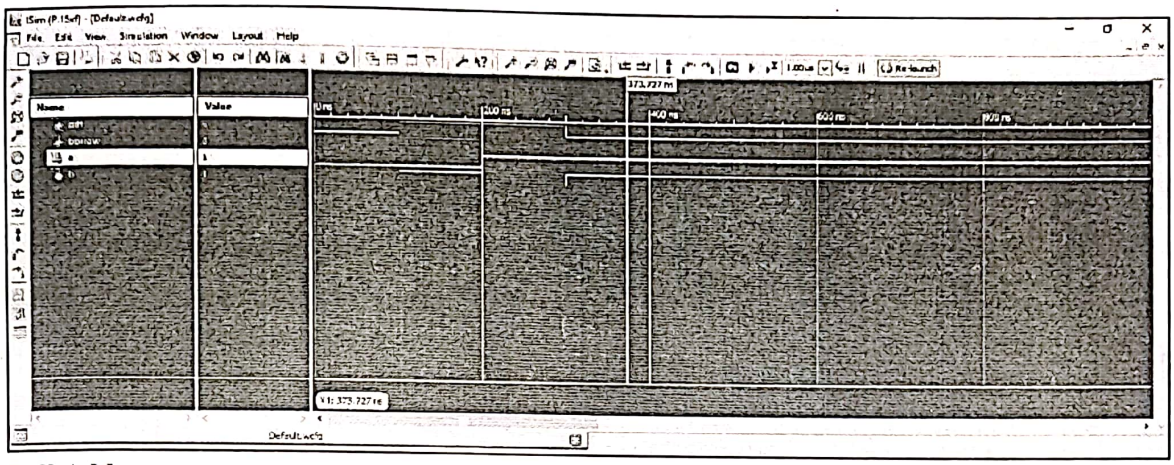
Simulation Output:

Half Adder-

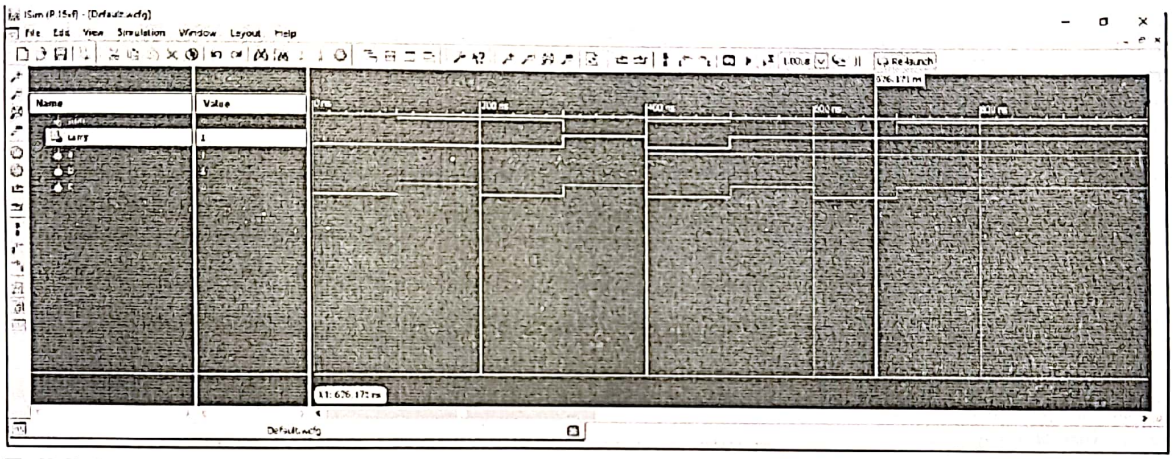


Scanned with OKEN Scanner

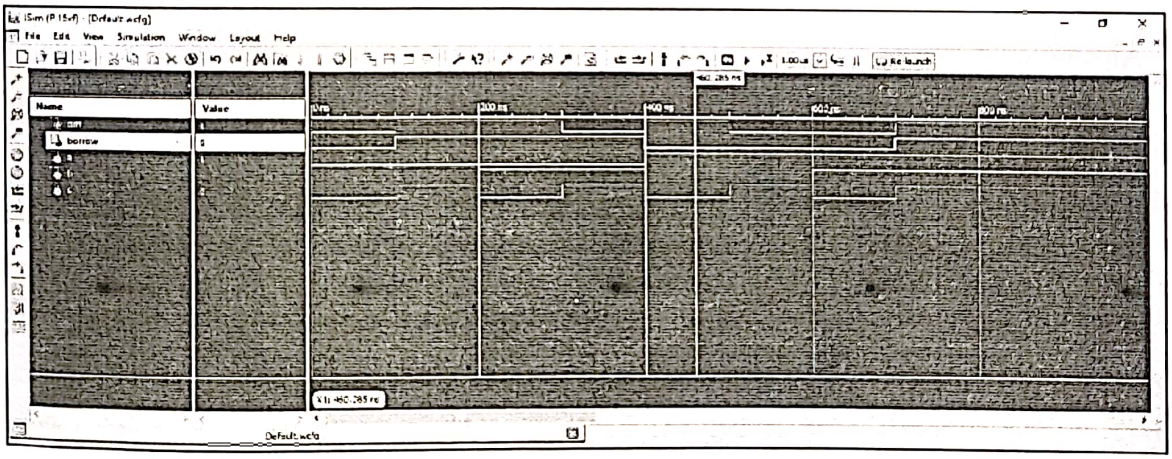
Half Substractor-



Full Adder-



Full Substractor



Results & conclusions: The Verilog code for adders and subtractors are verified.

Possible viva questions:

1. **What is the basic function of a binary adder?**

Ans: A binary adder combines two binary numbers, producing their sum and a possible carry output.

2. **What is a Half Adder in binary arithmetic?**

Scanned with OKEN Scanner

Ans: A Half Adder adds two binary digits (bits) and produces a sum and a carry-out. It lacks the ability to handle a carry-in from a previous stage.

3. How does a Full Adder differ from a Half Adder?

Ans: A Full Adder adds three binary digits (two inputs and a carry-in) to produce a sum and a carry-out, enabling it to handle carry inputs from previous stages.

4. What is a Half Subtractor in binary arithmetic?

Ans: A Half Subtractor subtracts two binary digits and produces a difference and a borrow-out. It does not consider borrow inputs from previous stages.

5. How does a Full Subtractor differ from a Half Subtractor?

Ans: A Full Subtractor subtracts three binary digits (two inputs and a borrow-in) to produce a difference and a borrow-out. It can handle borrow inputs from previous stages.

6. How does a Half Subtractor handle the case when the minuend is smaller than the subtrahend?

Ans: In such cases, the Half Subtractor produces a difference without a borrow-out, signifying that no borrowing is required for the subtraction.

7. Explain the significance of the carry-in and borrow-in signals in Full Adders and Full Subtractors.

Ans: The carry-in (Cin) in Full Adders and the borrow-in in Full Subtractors represent inputs from the previous stage, allowing these circuits to operate on multiple bits.

8. Can a Full Subtractor be constructed using Half Subtractors?

Ans: Yes, a Full Subtractor can be built using two Half Subtractors and additional logic to manage borrow inputs.

9. How does the carry propagation differ in Adders and Subtractors?

Ans: In Adders, carries propagate from lower bits to higher bits during addition, while in Subtractors, borrows propagate from higher bits to lower bits during subtraction.

10. Why is it called a "Half" Adder or Subtractor?

Ans: It is called "Half" because it handles only two input bits, not considering any carry or borrow inputs from previous stages, making it a simpler building block compared to "Full" Adders and Subtractors.

Scanned with OKEN Scanner

EXPERIMENT NO – 05

Design Verilog HDL to implement Decimal adder

Aim: To Design Verilog HDL to implement Decimal adder.

Theory: BCD or Binary coded decimal is a way of representing decimal digits in binary form. Generally 4 bits are used to represent values 0 to 9.

Decimal Digit	BCD 8 4 2 1
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0

Decimal Digit	BCD 8 4 2 1
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1

A BCD adder, takes in two BCD numbers as inputs, and outputs a BCD digit along with a carry output. If the sum of the BCD digits is less than or equal to 9, then we don't have a problem. But if its greater than 9, we have to convert it into BCD format. This is done by adding 6 to the sum and taking only the least significant 4 bits. The MSB bit is output as carry.

ACET, Dept. of ECE,

Consider the below BCD addition:

$$1001 + 1000 = 10001$$

$$9 + 8 = 17$$

The output 17 is more than 9. So we add 6 to it. So we get,

$$17+6 = 23 \text{ (in binary 23 is 10111)}$$

Now the least significant 4 bits (which is "0111") represent the units digit and the MSB(4th bit which is '1') bit represents the tens digit. Since the range of input is 0 to 9, the maximum output is 18. If you consider a carry it becomes 19. This means at the output side we need a 4 bit sum and a 1 bit carry to represent the most significant digit. These binary numbers are listed below

The condition for a correction and an output carry can be expressed by the Boolean Function

$$C = K + Z_8Z_4 + Z_8Z_2$$

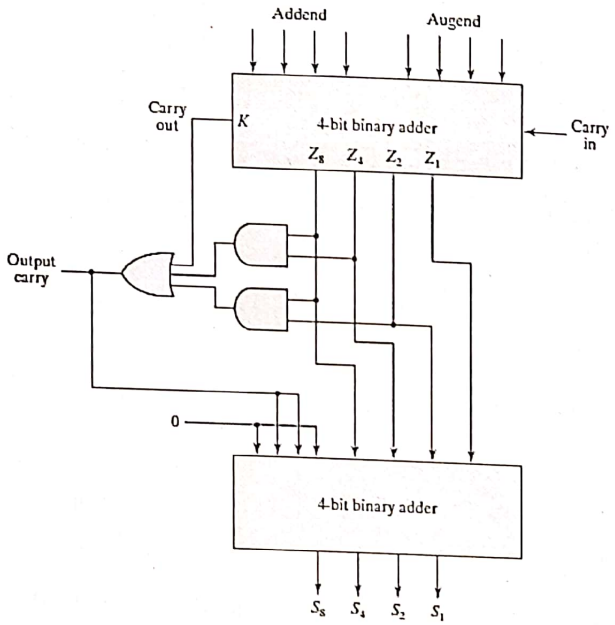
When C = 1, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage. A BCD adder that adds two BCD digits and produces a sum digit in BCD is shown in Figure below.

Scanned with OKEN Scanner

Derivation of BCD Adder

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

Logic Diagram:



For multiple digit addition, you can connect the carry_out to the carry input of the next adder. A simple cascading network of these small adders is enough to realize the multiple digit BCD addition.

Equipment:

- Computer with Modelsim SoftwareSpecifications:
- HP Computer P4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Softwares: Modelsim - 5.7c, Xilinx - 14.1i.

Procedure:

1. Open a Xilinx IDE, close old projects.
2. Create a new project through the project navigator icon.

Scanned with OKEN Scanner

3. Add VHDL/Verilog new source to the project depending on the user requirement.
4. Select the Verilog HDL mode.
5. Assign the inputs and outputs for the system to design.
6. And write the code as given below for the functionality.
7. Synthesize the code and correct the syntax errors if any.
8. Isim Simulator and Behavioral check syntax the Simulation behavioral model.
9. Observe the output timing waveform and verify it with the truth table.

Verilog code for BCD addition-Behavioral level:

```
module deciadder(a,b,carry_in,sum,carry);
                          //declare the inputs and outputs of the module with their sizes.
  input [3:0] a,b;
  input carry_in;
  output [3:0] sum;
  output carry;
                                  //Internal variables
  reg [4:0] sum_temp;
  reg [3:0] sum;
  reg carry;
                                  //always block for doing the addition
  always @(a,b,carry_in)
  begin
       sum_temp = a+b+carry_in;                    //add all the inputs
   if(sum_temp > 9)
      begin
      sum_temp = sum_temp+6;                       //add 6, if result is more than 9.
      carry = 1; //set the carry output
      sum = sum_temp[3:0];
      end
    else
      begin
      carry = 0;
      sum = sum_temp[3:0];
     end
  end
endmodule
```

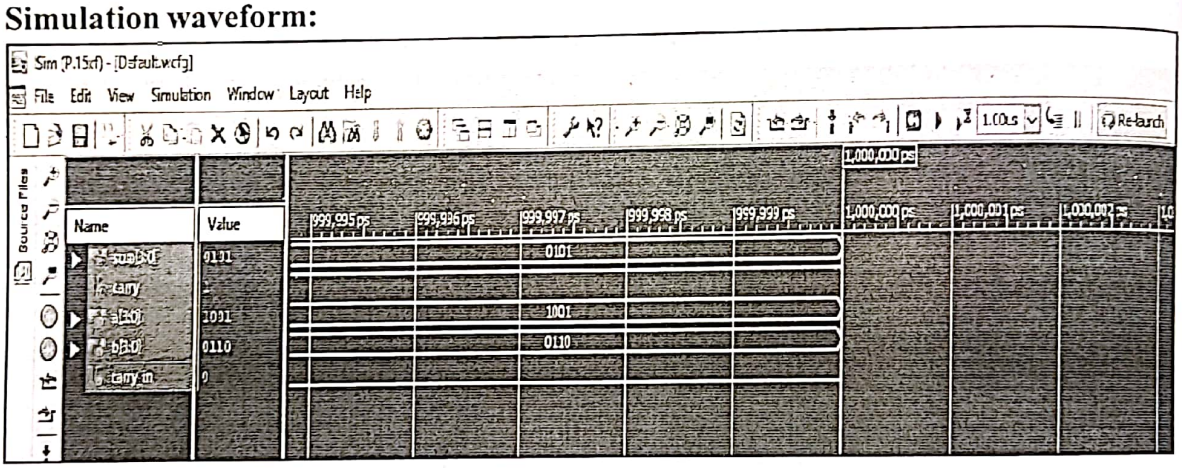
Verilog Testbench code for BCD adder:

```
module deciadder_tb;
      reg [3:0] a;
      reg [3:0] b;
      reg carry_in;
      wire [3:0] sum;
```


Scanned with OKEN Scanner

```
        wire carry;
    deciadder uut (.a(a), .b(b), .carry_in(carry_in), .sum(sum), .carry(carry));
        initial begin
            // Initialize Inputs
            a = 9;
            b = 6;
            carry_in = 0;
            #100;
        end
    endmodule
```

Simulation waveform:



Results & conclusions: The Verilog code for Decimal adder is simulated and verified.

Possible viva questions:

1. **What is a decimal adder?**
   Ans: A decimal adder is a digital circuit designed to perform addition operations specifically for decimal numbers.

2. **How does a decimal adder differ from a binary adder?**
   Ans: While binary adders operate on binary numbers, decimal adders process numbers in base-10, considering the decimal digits 0 through 9.

3. **Explain the significance of a carry in a decimal adder.**
   Ans: In a decimal adder, a carry occurs when the sum of two digits exceeds 9, requiring an additional unit to be carried to the next higher decimal place.

4. **How is overflow handled in a decimal adder?**
   Ans: Overflow occurs when the sum in a decimal place exceeds 9. In such cases, the carry is propagated to the next higher decimal place to prevent data loss.

Scanned with OKEN Scanner

5. **What are the applications of a decimal adder in digital systems?**
   Ans: Decimal adders are essential components in applications like financial calculations, where decimal numbers are prevalent, ensuring accurate arithmetic operations in electronic systems.

6. **How does the addition process differ between binary and decimal adders?**
   Ans: Decimal addition involves carrying over to the next decimal place when the sum exceeds 9, while binary addition carries over when the sum exceeds 1.

7. **Can a binary adder be adapted for decimal addition?**
   Ans: Yes, but it may not efficiently handle decimal-specific carry propagation. Decimal adders are optimized for base-10 arithmetic, considering the unique carry requirements of decimal digits.

8. **Explain the role of the carry-out in a decimal adder.**
   Ans: The carry-out signal indicates whether there is a carry from the most significant digit (MSD), signaling potential overflow in multi-digit decimal additions.

9. **How is subtraction implemented in a decimal adder circuit?**
   Subtraction in a decimal adder is achieved by using additional logic to complement and adjust the inputs based on the borrow generated during the subtraction process.

10. **Can a decimal adder handle negative decimal numbers?**
    Yes, by incorporating logic to manage sign bits and handling subtraction appropriately, a decimal adder can process negative decimal numbers.

Scanned with OKEN Scanner

EXPERIMENT NO – 06

Design Verilog program to implement Different types of multiplexer like 2:1, 4:1 and 8:1.

Aim: Design Verilog program to implement Different types of multiplexers like 2:1, 4:1 and 8:1.

Objectives:

1. Develop Verilog program for 2:1, 4:1, 8:1 multiplexer design.
2. Validate through simulation for accuracy and efficiency in circuit implementation.

Theory:

- A multiplexer (or data selector, abbreviated as MUX) has a group of data inputs (2^n) and a group of control inputs (n) (also called as select inputs).
- The control inputs are used to select one of the data inputs and connect it to the output terminal.

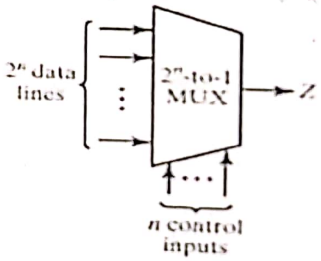


Figure 6.1: General block diagram of 2^n : 1 multiplexer

- A general block diagram of 2^n : 1 multiplexer is shown in the fig.3.6.
- A 2 to 1 multiplexer requires 1 select input, 4 to 1 multiplexers require 2 select inputs and 8 to 1 multiplexers require 3 select inputs.

2:1 Multiplexer

- A 2 to 1 multiplexer has 2 data inputs, 1 select input and 1 output.

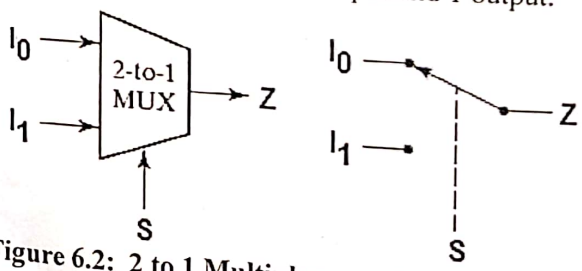


Figure 6.2: 2 to 1 Multiplexer and Switch Analog

- When the select (control) input A is 0, the switch is in the upper position and the MUX output is $Z = I_0$.
- When the select (control) input A is 1, the switch is in the lower position and the MUX output is $Z = I_1$.
- In other words, a MUX acts like a switch that selects one of the data inputs (I_0 or I_1) and transmits it to the output.

Scanned with OKEN Scanner

Truth Table:

Select (S)	tput (Z)
0	I_0
1	I_1

- The logic equation for the 2-to-1 MUX can be written as:

$$Z = \bar{S}I_0 + SI_1$$

Logic Diagram:

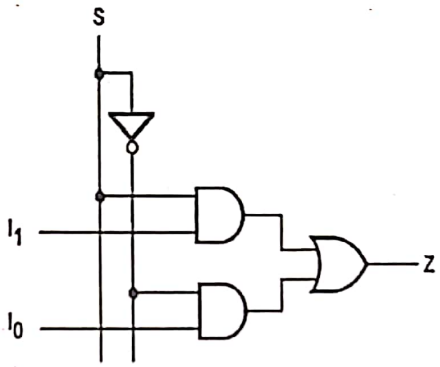


Figure 6.3: Logic diagram of 2:1 Multiplexer

4:1 Multiplexer

- A 4 to 1 multiplexer has 4 data inputs, 2 select inputs and 1 output.
- The 4 to 1 MUX acts like a four-position switch that transmits one of the four inputs to the output.

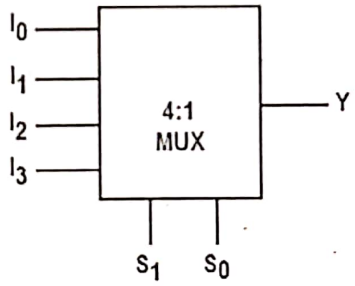


Figure 6.4: 4 to 1 Multiplexer

- Two select (control) inputs (S_1 and S_0) are needed to select one of the four inputs. If the control inputs are $S_1S_0 = 00$, the output is I_0; similarly, for the control inputs 01, 10, and 11 give outputs of I_1, I_2, and I_3, respectively.

Truth Table:

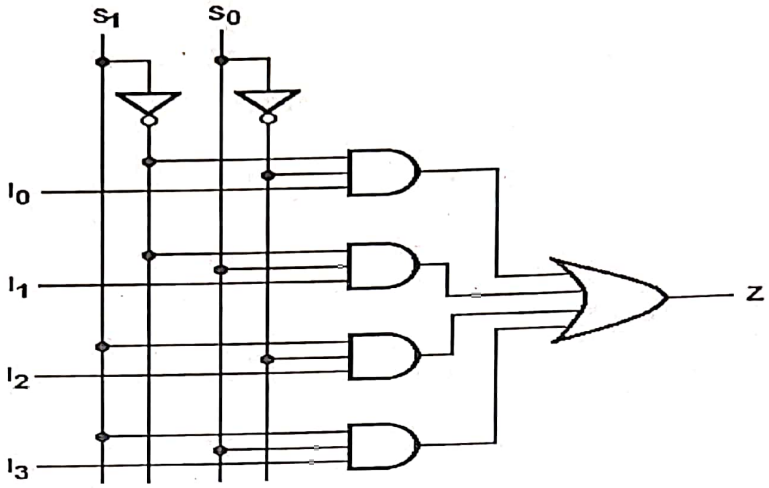
elect (S_1)	Select (S_0)	Output (Z)
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

- The logic equation for 4-to-1 multiplexer can be written as

$$Z = \bar{S}_1\bar{S}_0I_0 + \bar{S}_1S_0I_1 + S_1\bar{S}_0I_2 + S_1S_0I_3$$

Scanned with OKEN Scanner

Logic Diagram:



8:1 Multiplexer

- An 8 to 1 multiplexer has 8 data inputs, 3 select inputs and 1 output. 8 to 1 MUX selects one of eight data inputs using three select inputs

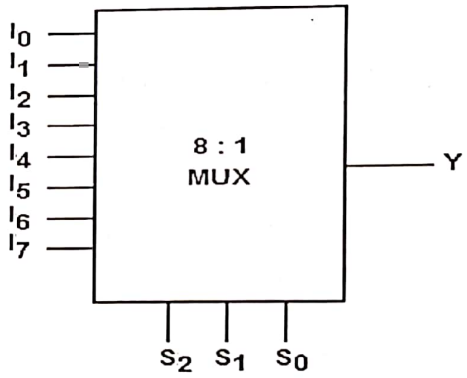


Figure 6.5: 8 to 1 Multiplexer

- The 8-to-1 MUX acts like an eight-position switch that transmits one of the eight inputs to the output.

Truth Table:

elect (S_2)	Select (S_1)	Select (S_0)	Output (Z)
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

- The logic equation for the 8 to1 MUX can be written as:

Scanned with OKEN Scanner

$$Z = \bar{S}_2\bar{S}_1\bar{S}_0 I_0 + \bar{S}_2\bar{S}_1 S_0 I_1 + \bar{S}_2 S_1\bar{S}_0 I_2 + \bar{S}_2 S_1 S_0 I_3 + S_2\bar{S}_1\bar{S}_0 I_4 + S_2\bar{S}_1 S_0 I_5 + S_2 S_1\bar{S}_0 I_6 + S_2 S_1 S_0 I_7$$

Equipment / components required:

- Computer with Xilinx - 14.7i. SoftwareSpecifications:
- HP Computer i4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Softwares: Xilinx - 14.7i.

Procedure / activity:

1. Open a Xilinx IDE, close old projects.
2. Create a new project through the project navigator icon.
3. Add VHDL/Verilog new source to the project depending on the user requirement.
4. Select the Verilog HDL mode.
5. Assign the inputs and outputs for the system to design.
6. And write the code as given below for the functionality.
7. Synthesize the code and correct the syntax errors if any.
8. Isim Simulator and Behavioral check syntax the Simulation behavioral model.
9. Observe the output timing waveform and verify it with the truth table.

Verilog code:

2:1 MUX

```
module two_to_one_MUX
(I0,I1,S,Y);
  input I0;
  input I1;
  input S;
  output Y;
        wire S_bar,w1,w2;
        not g1(S_bar,S);
        and (w1,I0,S_bar);
        and (w2,I1,S);
        or (Y,w1,w2);
endmodule
```

4:1 MUX

```
module four_to_one_MUX
(I1,I2,I3,I4,S1,S2,Y);
   input I1,I2,I3,I4;
   input S1,S2;
   output Y;

 assign
Y=((!S1)&(!S2)&I1)|((!S1
)&(S2)&I2)|(S1&(!S2)&I3
)|(S1&S2&I4);
endmodule
```

8:1 MUX

```
module eight_to_one_MUX
(I1,I2,I3,I4,I5,I6,I7,I8,S1,S2,S3,Y);
   input I1,I2,I3,I4,I5,I6,I7,I8;
   input S1,S2,S3;
   output Y;

assign  Y=((!S1)&(!S2)&(!S3)&I1)
|((!S1)&(!S2)&S3&I2)|((!S1)&S2&
(!S3)&I3)|((!S1)&S2&S3&I4)|(S1
&(!S2)&(!S3)&I5)|(S1&(!S2)&S3
&I6)|(S1&S2&(!S3)&I7)|(S1&S2&
S3&I8);
endmodule
```

Verilog Testbench code:

2:1 MUX

```
module two_to_one_MUX_tb;
        reg I0;
        reg I1;
        reg S;
        wire Y;
two_to_one_MUX uut (
                .I0(I0),
```

4:1 MUX

```
module four_to_one_MUX_tb;
        reg I1;
        reg  I2;
        reg  I3;
        reg I4;
        reg S1;
        reg S2;
```

8:1 MUX

```
module eight_to_one_MUX1;
        reg I1;
        reg I2;
        reg I3;
        reg I4;
        reg I5;
        reg I6;
```


Scanned with OKEN Scanner

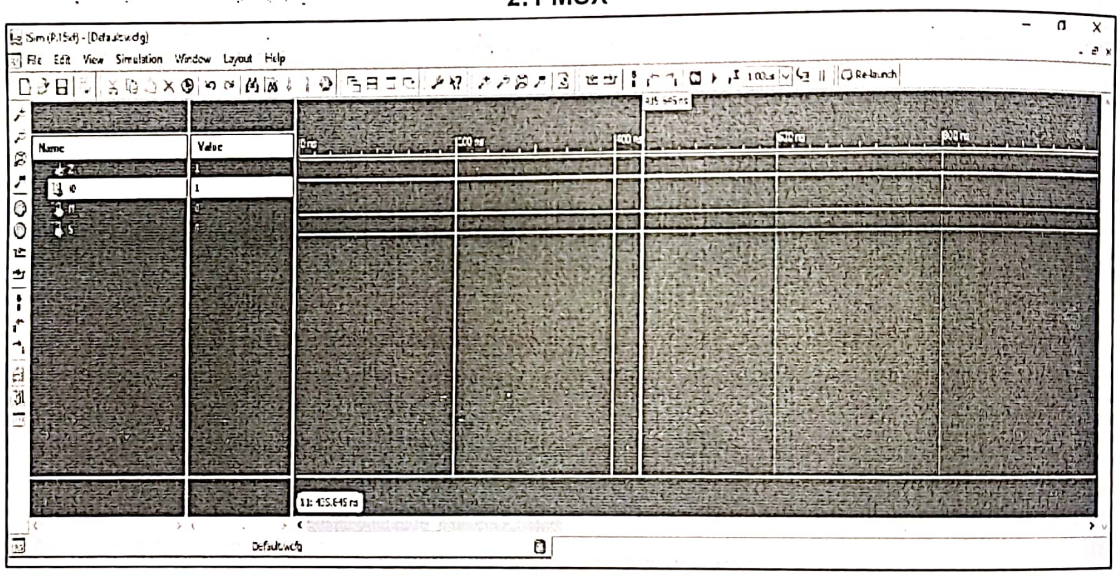
```
        .I1(I1),
        .S(S),
        .Y(Y) );
    initial begin
I0 = 1; I1 = 0;
S = 0; #100;
    end
endmodule
```

```
    wire Y;
four_to_one_MUX uut (
        .I1(I1),
        .I2(I2),
        .I3(I3),
        .I4(I4),
        .S1(S1),
        .S2(S2),
        .Y(Y) );
    initial begin
I1 = 0;I2 = 0;I3 = 0;I4 = 1;
S1 = 0;S2 = 1; #100;
    end
endmodule
```

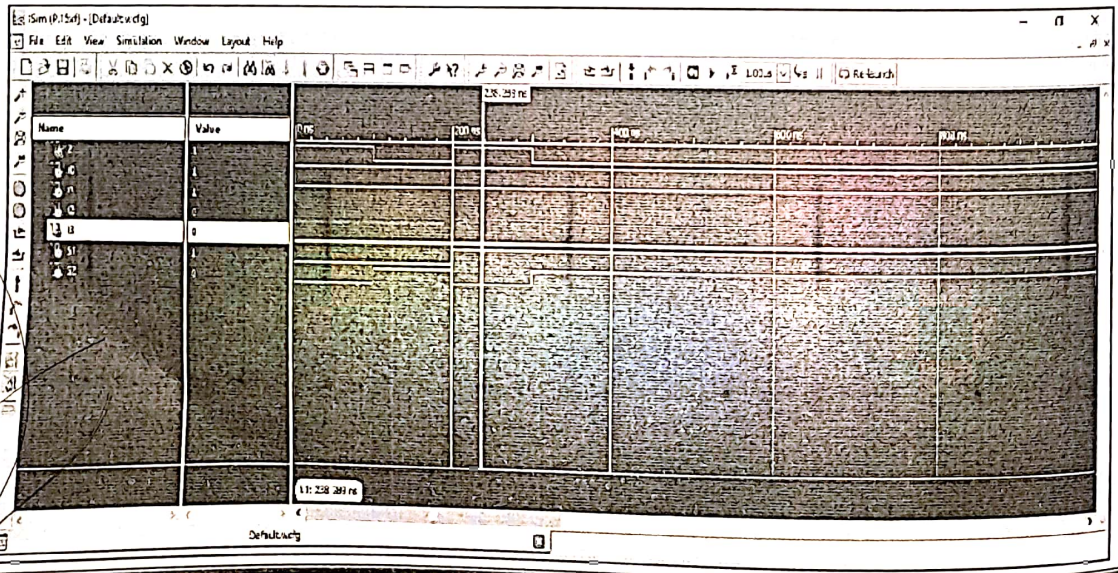
```
    reg I7;
    reg  I8;
    reg  S1;
    reg  S2;
    reg  S3;
    wire Y;
eight_to_one_MUX uut (
.I1(I1), .I2(I2), .I3(I3), .I4(I4),
.I5(I5), .I6(I6), .I7(I7), .I8(I8),
.S1(S1), .S2(S2), .S3(S3),
.Y(Y) );
  initial begin
I1 = 1;I2 = 1;I3 = 1; I4 = 1;I5 =
1;I6 = 0;I7 = 1;I8 = 1;
S1 = 1; S2 = 0; S3 = 1; #100;
    end
endmodule
```

Simulation Output:

2:1 MUX

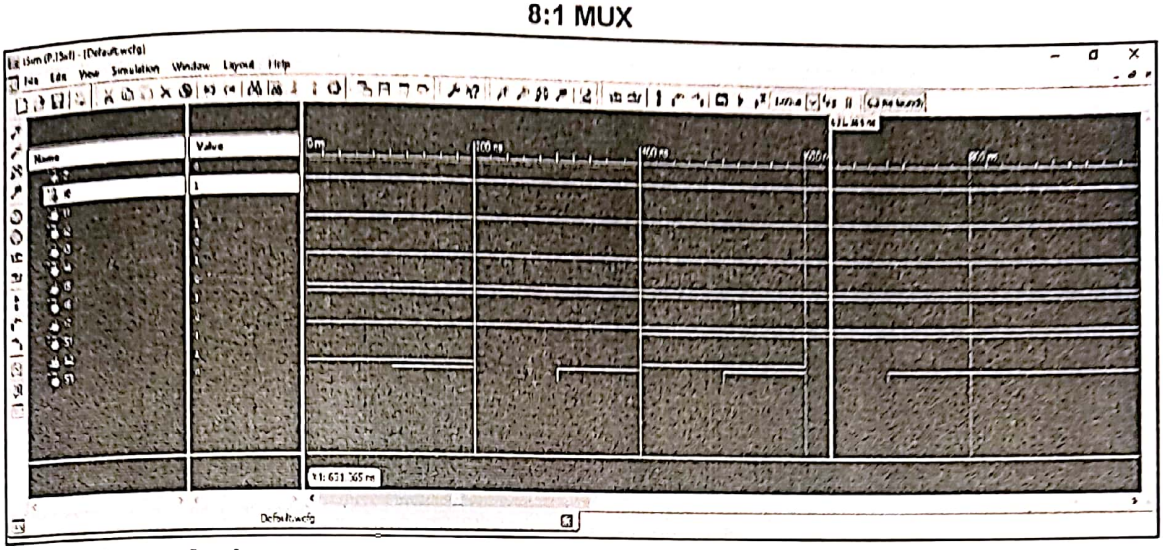


4:1-MUX



Scanned with OKEN Scanner

8:1 MUX



Results & conclusions:

The Verilog code for Different types of multiplexers like 2:1, 4:1 and 8:1 is simulated and verified.

Possible viva questions:

1. **What is the purpose of a multiplexer?**

Ans: A multiplexer is a digital circuit that selects one of several input signals and directs it to a single output.

2. **How does a 2:1 multiplexer differ from a 4:1 multiplexer?**

Ans: A 2:1 multiplexer has two inputs, and a 4:1 multiplexer has four inputs.

3. **Explain the role of the select input in a multiplexer.**

Ans: A 2:1 multiplexer has two inputs, and a 4:1 multiplexer has four inputs.

4. **What happens when the select input is 0 in a 2:1 multiplexer?**

Ans: When the select input is 0 in a 2:1 multiplexer, the output is connected to the first input.

5. **How is the output determined in an 8:1 multiplexer?**

Ans: The select input in an 8:1 multiplexer specifies which of the eight inputs is routed to the output.

6. **Can a multiplexer be used to implement logic gates? How?**

Ans: Yes, multiplexers can be configured to function as logic gates by appropriately selecting inputs.

7. **Can Verilog code for a 2:1 multiplexer be reused for a 4:1 multiplexer? Why or why not?**

Ans: Some code can be reused, but adjustments are needed for the increased number of inputs.

8. **How does the complexity of a multiplexer circuit increase with the number of inputs?**

Ans: The complexity of a multiplexer circuit increases linearly with the number of inputs, leading to more logic gates and connections.

9. **Explain the concept of data routing in multiplexers.**

Ans: Data routing refers to selecting and transmitting a specific input to the output based on the select input.

Scanned with OKEN Scanner

10. What is the difference between a multiplexer and a demultiplexer?

Ans: A multiplexer selects one input among many for output, while a demultiplexer routes one input to multiple outputs.

Scanned with OKEN Scanner

EXPERIMENT NO – 07

Design Verilog program to implement types of De-Multiplexer.

Aim:

To Design Verilog program to implement Different types of De-multiplexer like 1:2, 1:4 and 1:8.

Objectives:

1. Implement Verilog program for 1:2, 1:4, 1:8 Demultiplexer designs.
2. Verify functionality through simulation for accuracy and performance assessment.

Theory:

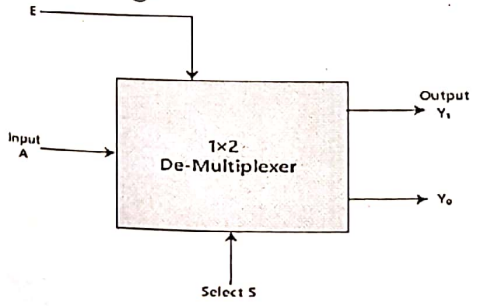
A De-multiplexer is a combinational circuit that has only 1 input line and 2^N output lines. Simply, the multiplexer is a single-input and multi-output combinational circuit. The information is received from the single input lines and directed to the output line. On the basis of the values of the selection lines, the input will be connected to one of these outputs. De-multiplexer is opposite to the multiplexer.

Unlike encoder and decoder, there are n selection lines and 2^n outputs. So, there is a total of 2^n possible combinations of inputs. De-multiplexer is also treated as **De-mux**.

There are various types of De-multiplexer which are as follows:

1×2 De-multiplexer: In the 1 to 2 De-multiplexer, there are only two outputs, i.e., Y_0, and Y_1, 1 selection lines, i.e., S_0, and single input, i.e., A. Based on the selection value, the input will be connected to one of the outputs. The block diagram and the truth table of the 1×2 multiplexer are given below.

Block Diagram:



Truth Table:

INPUTS	Output	
S_0	Y_1	Y_0
0	0	A
1	A	0

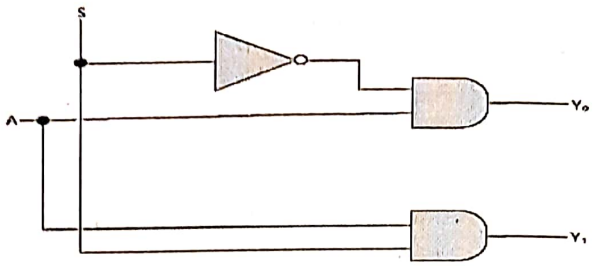
The logical expression of the term Y is as follows:

$$Y_0=S_0'.A$$

$$Y_1=S_0.A$$

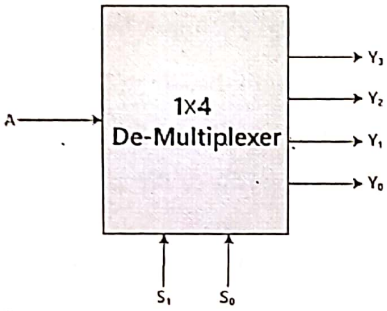
Logical circuit of the above expressions is given below:

Scanned with OKEN Scanner



1×4 De-multiplexer: In 1 to 4 De-multiplexer, there are total of four outputs, i.e., Y_0, Y_1, Y_2, and Y_3, 2 selection lines, i.e., S_0 and S_1 and single input, i.e., A. Based on the combination of inputs which are present at the selection lines S_0 and S_1, the input be connected to one of the outputs. The block diagram and the truth table of the 1×4 multiplexer is given below.

Block Diagram:



Truth Table:

INPUTS		Output			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	A
0	1	0	0	A	0
1	0	0	A	0	0
1	1	A	0	0	0

The logical expression of the term Y is as follows:

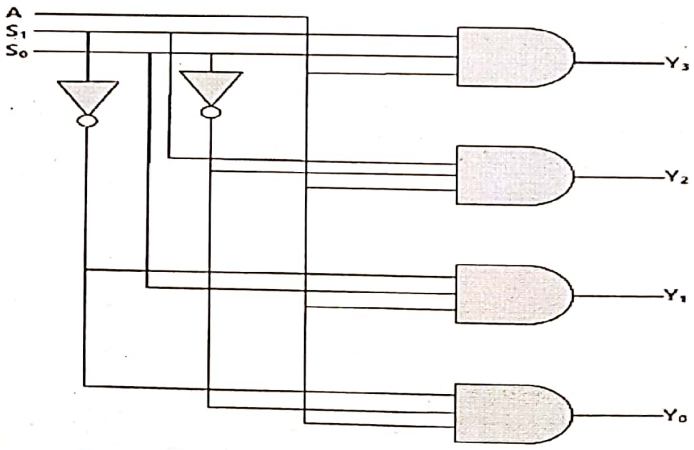
$Y_0 = S_1' S_0' A$

$y_1 = S_1' S_0 A$

$y_2 = S_1 S_0' A$

$y_3 = S_1 S_0 A$

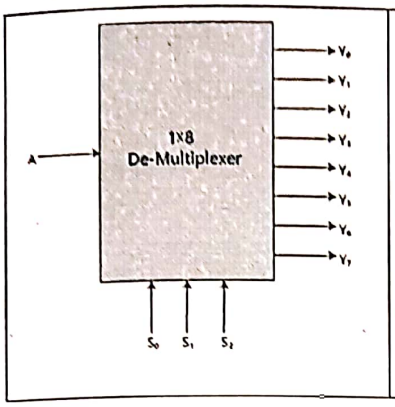
Logical circuit of the above expressions is given below:



1×8 De-multiplexer: In 1 to 8 De-multiplexer, there are total of eight outputs, i.e., Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7, 3 selection lines, i.e., S_0, $S_1$and S_2 and single input, i.e., A. On the basis of the combination of inputs which are present at the selection lines S^0, S^1 and S_2, the input will be connected to one of these outputs. The block diagram and the truth table of the 1×8 de-multiplexer is given below.

Block Diagram:	Truth Table:

Scanned with OKEN Scanner



INPUTS			Output							
S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	A
0	0	1	0	0	0	0	0	0	A	0
0	1	0	0	0	0	0	0	A	0	0
0	1	1	0	0	0	0	A	0	0	0
1	0	0	0	0	0	A	0	0	0	0
1	0	1	0	0	A	0	0	0	0	0
1	1	0	0	A	0	0	0	0	0	0
1	1	1	A	0	0	0	0	0	0	0

The logical expression of the term Y is as follows:

$Y_0=S_0'.S_1'.S_2'.A$

$Y_1=S_0.S_1'.S_2'.A$

$Y_2=S_0'.S_1.S_2'.A$

$Y_3=S_0.S_1.S_2'.A$

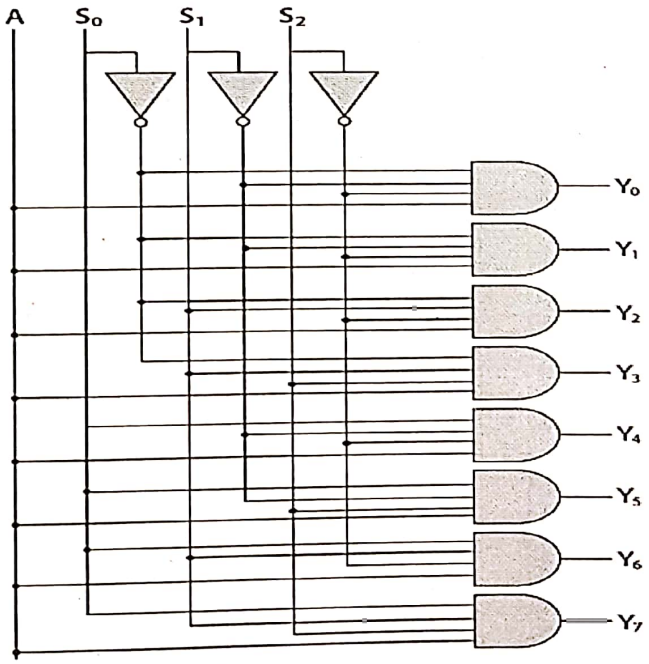
$Y_4=S_0'.S_1'.S_2\ A$

$Y_5=S_0.S_1'.S_2\ A$

$Y_6=S_0'.S_1.S_2\ A$

$Y_7=S_0.S_1.S_3.A$

Logical circuit of the above expressions is given below:



Equipment / components required:

- Computer with Xilinx - 14.7i.
- HP Computer i4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Software: Xilinx ISE v14.7 IDE

Scanned with OKEN Scanner

Procedure / activity:

1. Open a Xilinx IDE, close old projects.
2. Create a new project through the project navigator icon.
3. Add VHDL/Verilog new source to the project depending on the user requirement.
4. Select the Verilog HDL mode.
5. Assign the inputs and outputs for the system to design.
6. And write the code as given below for the functionality.
7. Synthesize the code and correct the syntax errors if any.
8. Isim Simulator and Behavioral check syntax the Simulation behavioral model.
9. Observe the output timing waveform and verify it with the truth table.

Verilog code:

1:2 DEMUX

```
module demux12_BM(S0,A,Y);
  input A, S0;
  wire A, S0;
  output [1:0] Y;
  reg [1:0] Y;
always @ (A or S0)
  begin
  case(S0)
  1'b0: Y={A,1'b0};
  default: Y={1'b0, A};
  endcase
  end
endmodule
```

1:4 DEMUX

```
module
one_four_demux_BM(S,A,Y);
input A;
wire A;
input [1:0] S;
wire [1:0] S;
output [3:0] Y;
reg [3:0] Y;

always @(A or S) begin

case(S)
1'b00: Y= {A,3'b000};
1'b01: Y= {1'b0, A,2'b00};
1'b10: Y= {2'b00, A,1'b0};
default: Y= {3'b000, A};
endcase
end
endmodule
```

1:8 DEMUX

```
module
one_eight_demux_BM(S,A,Y);
input A;
wire A;
input [2:0] S;
wire [2:0] S;
output [7:0] Y;
reg [7:0] Y;

always @(A or S)
begin

case(S)
0: Y={A,7'b0000000};
1: Y= {1'b0, A,6'b000000};
2: Y= {2'b00, A,5'b00000};
3: Y= {3'b000, A,4'b0000};
4: Y= {4'b0000, A,3'b000};
5: Y= {5'b00000, A,2'b00};
6: Y= {6'b000000, A,1'b0};
default: Y= {7'b0000000, A};
Endcase
End
Endmodule
```

Verilog Testbench code:

1:2 DEMUX

```
module demux12_BM_tb;
        reg S0;
        reg A;
        wire [1:0] Y;
demux12_BM uut (
                .S0(S0),
                .A(A),
                .Y(Y) );
```

1:4 DEMUX

```
module
one_four_demux_BMtb;
        reg [1:0] S;
        reg A;
        wire [3:0] Y;
one_four_demux_BM uut (
                .S(S),
                .A(A),
```

1:8 DEMUX

```
module
one_eight_demux_BMtb;
        reg [2:0] S;
        reg A;
        wire [7:0] Y;
one_eight_demux_BM uut (
                .S(S),
                .A(A),
```


Scanned with OKEN Scanner

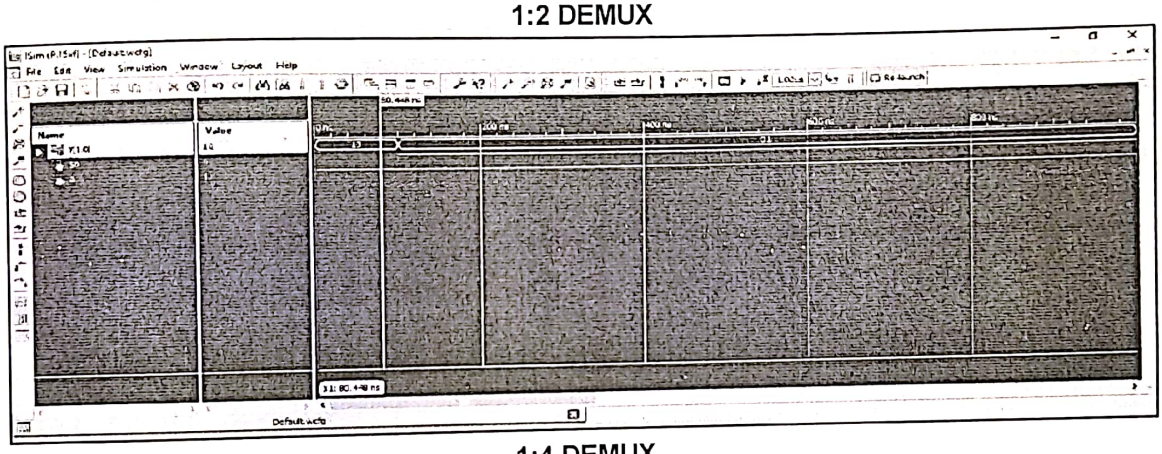
```
initial begin
  S0 = 0; A = 1; #100;
  S0 = 1; A = 1; #100;
end
endmodule
```

```
.Y(Y) );
initial begin
  S = 0; A = 1; #100;
  S = 1; A = 1; #100;
  S = 2; A = 1; #100;
  S = 3; A = 1; #100;
end
endmodule
```

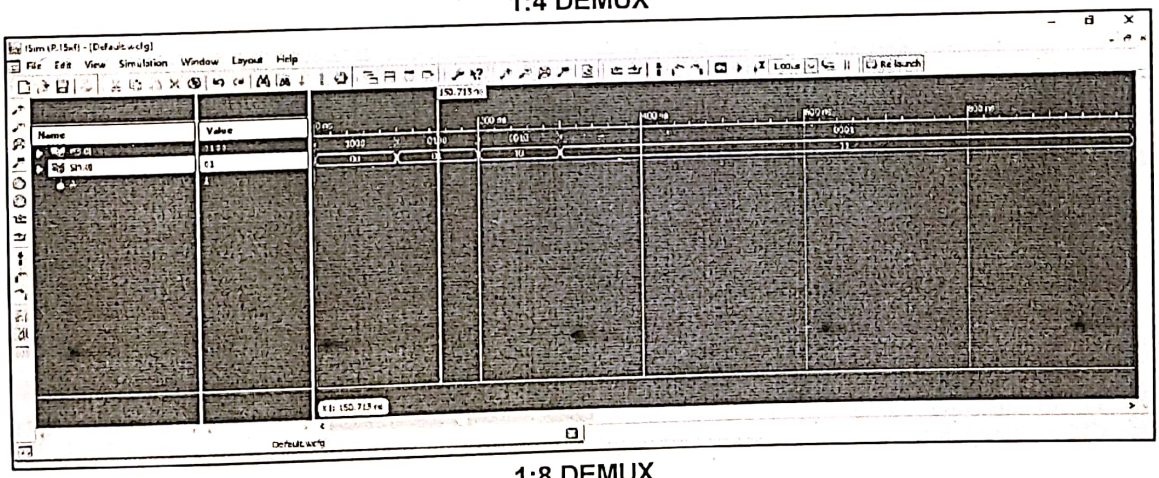
```
.Y(Y) );
initial begin
  S = 0; A = 1; #100;
  S = 1; A = 1; #100;
  S = 2; A = 1; #100;
  S = 3; A = 1; #100;
  S = 4; A = 1; #100;
  S = 5; A = 1; #100;
  S = 6; A = 1; #100;
  S = 7; A = 1; #100;
end
endmodule
```

Simulation Output:

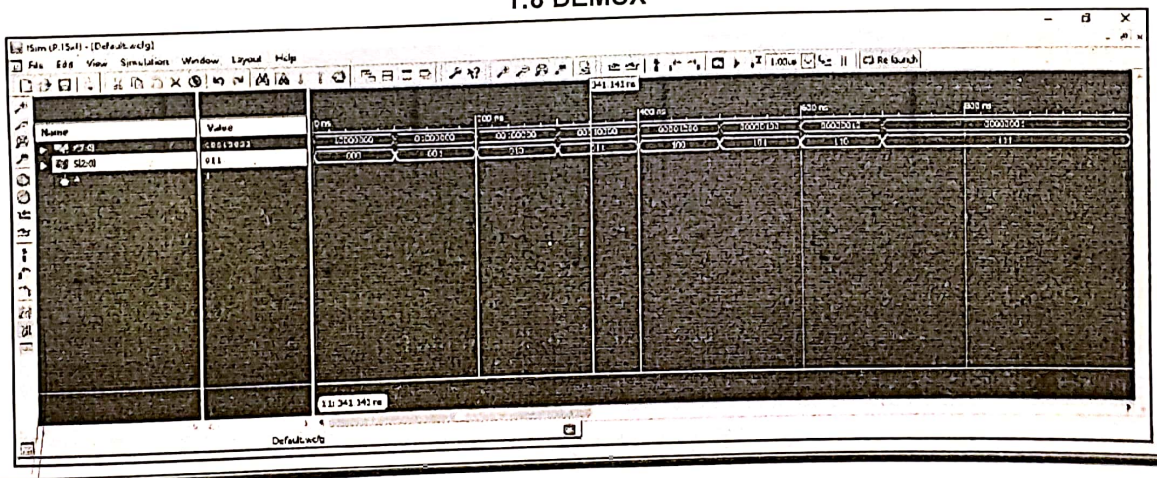
1:2 DEMUX



1:4 DEMUX



1:8 DEMUX



Scanned with OKEN Scanner

Results & conclusions:

The Verilog code for Different types of De-multiplexer like 1:2, 1:4 and 1:8 is simulated and verified.

Possible viva questions:

1. **What is the primary function of a De-Multiplexer (De-Mux)?**
Ans: A De-Mux takes a single input and directs it to one of several possible outputs.

2. **Explain the operation of a 1-to-2 De-Multiplexer.**
Ans: A 1-to-2 De-Mux selects between two output lines based on the control input.

3. **How does a 1-to-4 De-Multiplexer differ from a 1-to-2 De-Multiplexer?**
Ans: A 1-to-4 De-Mux directs one input to one of four outputs based on the control input.

4. **What is the significance of the Enable input in a De-Multiplexer?**
Ans: The Enable input allows or inhibits the De-Mux operation, controlling the output based on its state.

5. **Discuss the concept of a Binary De-Multiplexer.**
Ans: A Binary De-Mux with n control lines can select one of 2^n output lines for the input.

6. **How does a De-Multiplexer with multiple outputs, like 1-to-8, function?**
Ans: A 1-to-8 De-Mux routes a single input to one of eight possible output lines determined by the control inputs.

7. **What is the purpose of a Priority De-Multiplexer?**
Ans: A Priority De-Mux ensures that a specific input is directed to its corresponding output in the presence of multiple inputs.

8. **In what scenarios is a Decoder considered a specialized De-Multiplexer?**
Ans: A Decoder acts as a De-Mux by converting a binary code into an equivalent decimal or other code.

9. **How do De-Multiplexers contribute to data routing in communication systems?**
Ans: De-Muxes route data to specific channels, aiding in the distribution of information in communication networks.

10. **Explain how De-Multiplexers enhance flexibility in digital circuits.**
Ans: De-Muxes provide flexibility by enabling the selection of different output lines for a single input based on control signals.

Scanned with OKEN Scanner

EXPERIMENT NO – 08

Design Verilog program for implementing various types of Flip-Flops such as SR, JK and D

Aim: To design and simulate Verilog modules for three different types of flip-flops (SR, JK, and D) using behavioral modeling. The program should demonstrate the functionality and behavior of these flip-flops under various input conditions.

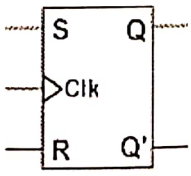
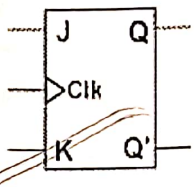
Objectives:

- Implement Verilog modules for SR, JK, and D flip-flops using behavioral modeling. Develop a testbench to simulate the behavior of the implemented flip-flops.
- Apply different input conditions to the flip-flops in the testbench to observe their responses.
- Verify that the flip-flops exhibit the expected sequential logic behavior, such as state changes and toggling.

Theory:

Flip-flops are synchronous bitable devices. The term synchronous means the output changes state only when the clock input is triggered. That is, changes in the output occur in synchronization with the clock. A flip-flop circuit has two outputs, one for the normal value and one for the complement value of the stored bit. Since memory elements in sequential circuits are usually flip-flops, it is worth summarizing the behavior of various flip-flop types before proceeding further. Flip-flops can be divided into four basic types: SR, JK, D and T. They differ in the number of inputs and in the response invoked by different value of input signals. The four types of flip-flops are defined in the Table 8.1. Each of these flip-flops can be uniquely described by its graphical symbol, its characteristic table, its characteristic equation or excitation table. All flip-flops have output signals Q and Q'.

Table 8.1: Flip-flops and their properties

<table>
<tr><th>Flip-Flop Name</th><th>Flip-Flop Symbol</th><th>Characteristic Table</th><th>Characteristic Equation</th><th>Excitation Table</th></tr>
<tr><td>SR</td><td> 
</td><td>
<table>
<tr><th>S</th><th>R</th><th>Q(next)</th></tr>
<tr><td>0</td><td>0</td><td>Q</td></tr>
<tr><td>0</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>?</td></tr>
</table>
</td><td>Q(next) = S + R'Q
SR = 0</td><td>
<table>
<tr><th>Q</th><th>Q(next)</th><th>S</th><th>R</th></tr>
<tr><td>0</td><td>0</td><td>0</td><td>X</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>X</td><td>0</td></tr>
</table>
</td></tr>
<tr><td>JK</td><td> 
</td><td>
<table>
<tr><th>J</th><th>K</th><th>Q(next)</th></tr>
<tr><td>0</td><td>0</td><td>Q</td></tr>
<tr><td>0</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>Q'</td></tr>
</table>
</td><td>Q(next) = JQ' + K'Q</td><td>
<table>
<tr><th>Q</th><th>Q(next)</th><th>J</th><th>K</th></tr>
<tr><td>0</td><td>0</td><td>0</td><td>X</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>X</td></tr>
<tr><td>1</td><td>0</td><td>X</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>X</td><td>0</td></tr>
</table>
</td></tr>
</table>

Scanned with OKEN Scanner

D	D	Q	Clk	Q'	D	Q(next)	Q(next) = D	Q	Q(next)	D
					0	0		0	0	0
					1	1		0	1	1
								1	0	0
								1	1	1

Logic diagram

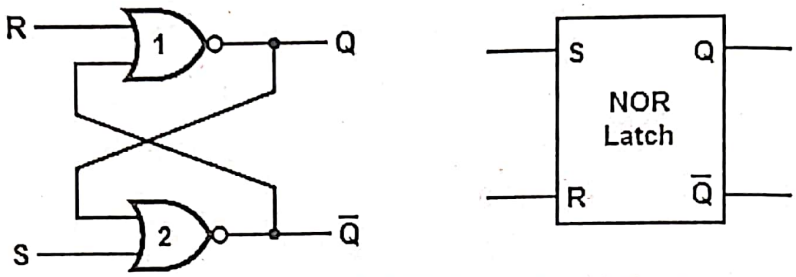


Figure 8.1: SR (a) NOR Gate latch (b) Symbol

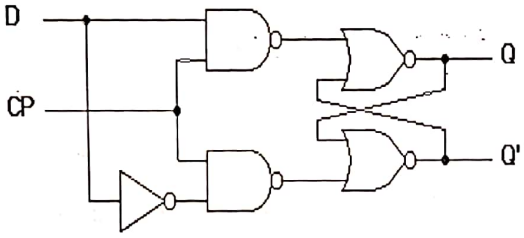


Figure 8.2: D- Flip Flop

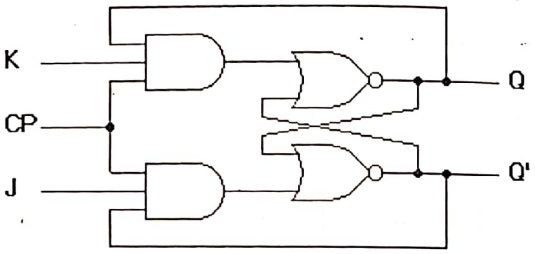


Figure 8.3: JK Flip Flop

Graphs

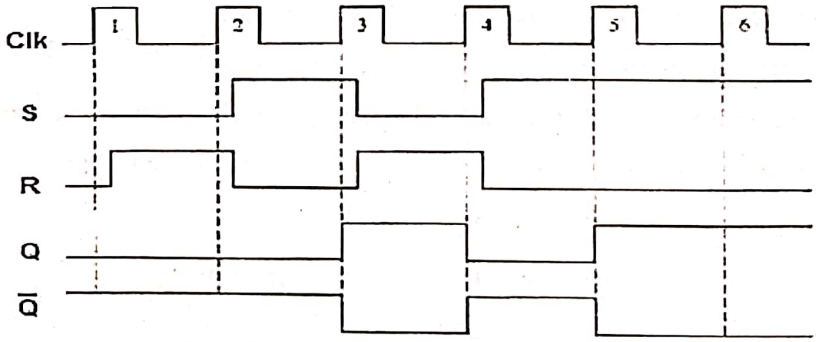


Figure 8.4: Timing diagram of SR Flip-Flop

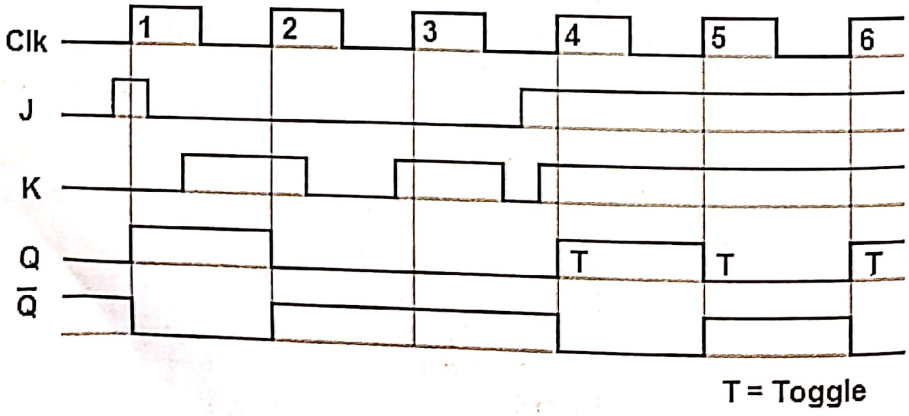


Figure 8.5: Timing diagram of JK Flip-Flop

Scanned with OKEN Scanner

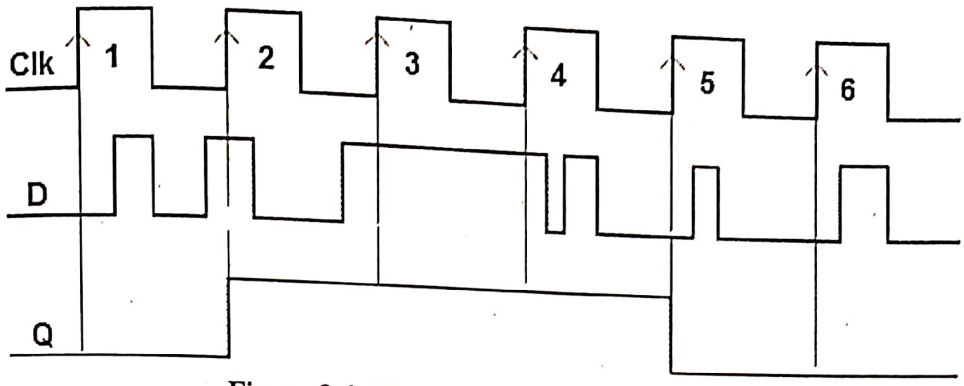


Figure 8.6: Timing diagram of D Flip-Flop

Equipment / components required:

- Computer with Modelsim SoftwareSpecifications:
- HP Computer P4 Processor – 2.8 GHz, 2GB RAM, 160 GB Hard Disk
- Softwares: Modelsim - 5.7c, Xilinx - 14.1i.

Procedure:

1. Open a Xilinx IDE, close old projects.
2. Create a new project through the project navigator icon.
3. Add VHDL/Verilog new source to the project depending on the user requirement.
4. Select the Verilog HDL mode.
5. Assign the inputs and outputs for the system to design.
6. And write the code as given below for the functionality.
7. Synthesize the code and correct the syntax errors if any.
8. Isim Simulator and Behavioral check syntax the Simulation behavioral model.
9. Observe the output timing waveform and verify it with the truth table.

Verilog code:

1. SR Flip-Flop

```
module srlax(
   input wire S,
   input wire R,
   input output reg Q,
   output wire Q_bar );
always @(S, R) begin
if (S && ~R)
        Q <= 1'b1;
else if (~S && R)
        Q <= 1'b0;
else if (S==0 && R==0)
        Q <= Q;
else if (S==1 && R==1)
        Q <= 1'bz;
      end
 assign Q_bar = ~Q;
    endmodule
```

2. JK Flip Flop

```
module JK_FF(
  input wire j,
  input wire k,
  output reg Q,
  output wire Q_bar );

  always @(j, k) begin
   if (j && ~k)
     Q <= 1'b1;
   else if (~j && k)
     Q <= 1'b0;
   else if (j==0 && k==0)
     Q <= Q;
        else if (j==1 && k==1)
     Q <= !Q;
  end
  assign Q_bar = ~Q;
```

3. D Flip Flop

```
module D_FF(
  input wire D,
  output reg Q,
  output wire Q_bar
);

  always @(D) begin
    if (D==0)
     Q <= 0;
        Else
     Q <= 1;
  End

  assign Q_bar = ~Q;
 Endmodule
```


Scanned with OKEN Scanner

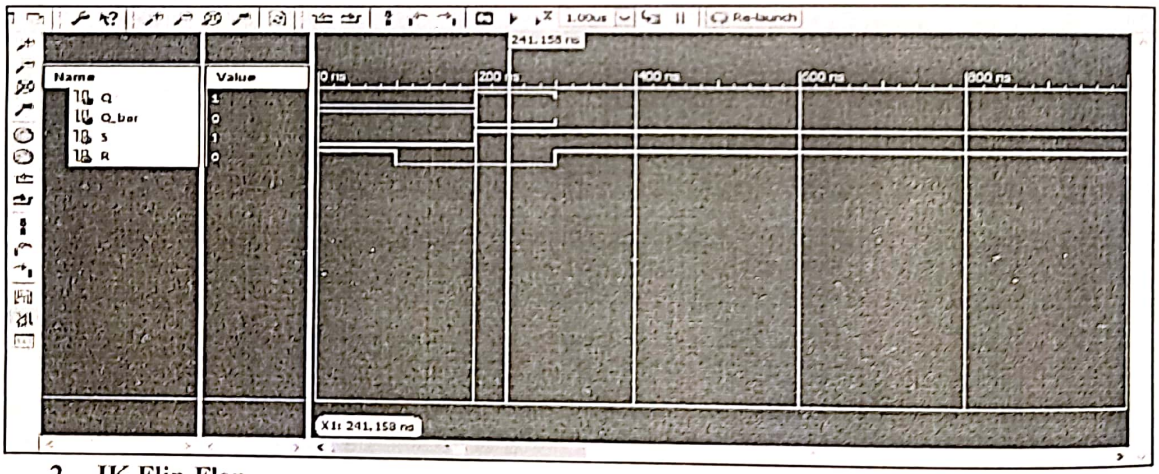
	endmodule	

Verilog Testbench code:

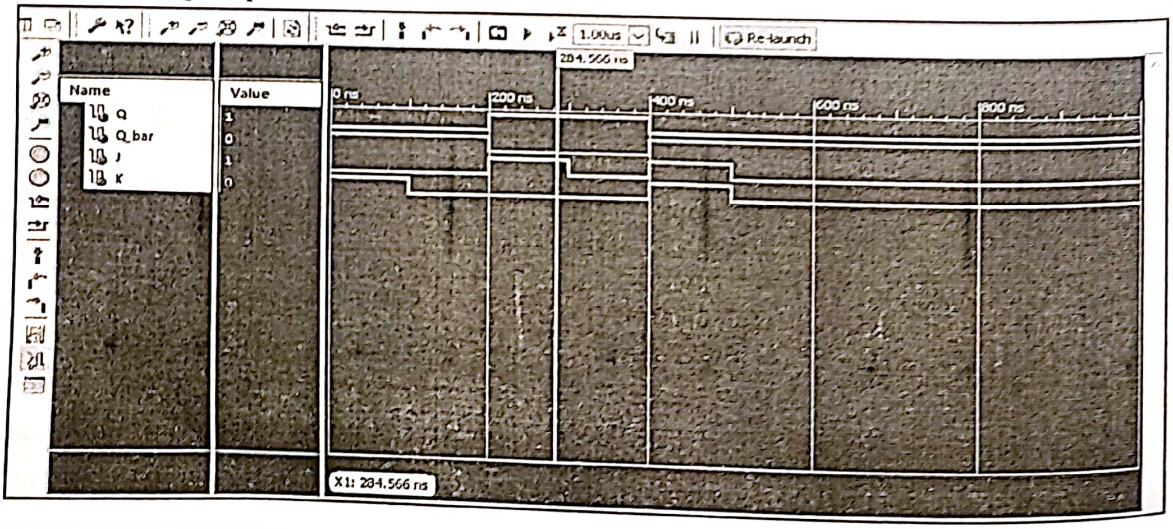
1. SR Flip-Flop:	2. JK Flip Flop	3. D Flip Flop
module srlax_tb; reg S; reg R; wire Q; wire Q_bar; srlax uut (.S(S), .R(R), .Q(Q), .Q_bar(Q_bar)); initial begin S = 0; R = 0; #100; S = 1; R = 0; #100; S = 0; R = 1; #100; S = 1; R = 1; #100; end endmodule	module JK_FF_tb; reg J; reg K; wire Q; wire Q_bar; JK_FF uut (.J(J), .K(K), .Q(Q), .Q_bar(Q_bar)); initial begin J = 0; K = 0 ;#100; J = 1; K = 0; #100; J = 0; K = 1 ;#100; J = 1; K = 1; #100; end endmodule	module D_FF_tb; reg D; wire Q; wire Q_bar; D_FF uut (.D(D), .Q(Q), .Q_bar(Q_bar)); initial begin D = 0; #100; D = 1; #100; end endmodule

Simulation Output:

1. SR Flip-Flop

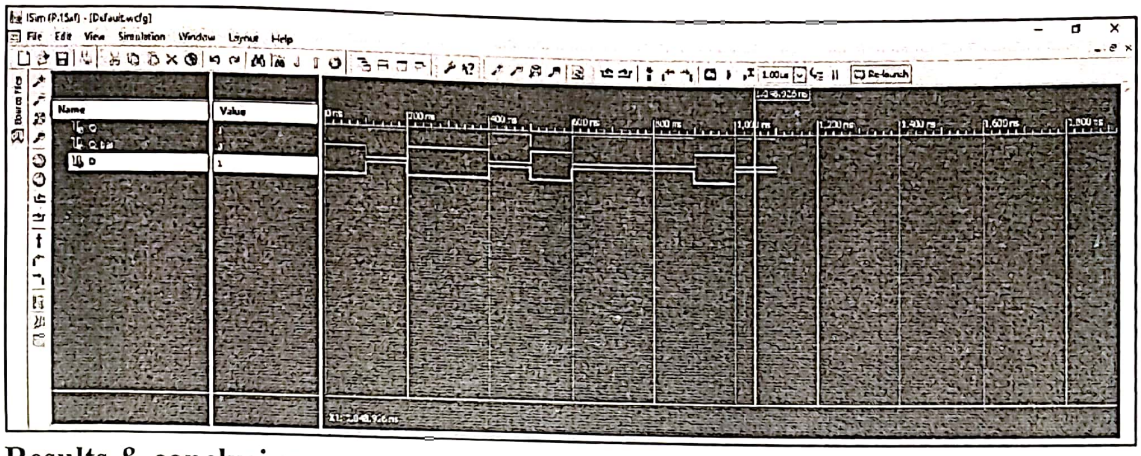


2. JK Flip Flop



Scanned with OKEN Scanner

3. **D Flip Flop**



Results & conclusions:

The Verilog code for Different types various types of Flip-Flops such as SR, JK and D.is simulated and verified

Possible viva questions:

1. **What is an SR flip-flop, and what are its primary characteristics?**
   An SR flip-flop is a digital circuit with Set (S) and Reset (R) inputs, storing binary information.
2. **What are the possible states of an SR flip-flop?**
   Possible SR flip-flop states: Set (Q=1), Reset (Q=0), Hold (Q unchanged).
3. **How does the presence of an illegal state in an SR flip-flop affect its operation?**
   Illegal states in SR flip-flop may lead to undefined behavior or undesired output transitions.
4. **Describe a real-world application where an SR flip-flop is useful.**
   SR flip-flops are used in memory circuits, latch circuits, and control systems for state storage.
5. **How can you use an SR flip-flop to create a simple toggle circuit?**
   Connect S and R inputs of an SR flip-flop, toggling between Set and Reset states.
6. **Explain the concept of setup time and hold time in the context of an SR flip-flop.**
   Setup time ensures stable inputs before a clock edge, while hold time maintains stability after the edge.
7. **What are the features that distinguish a JK flip-flop from an SR flip-flop?**
   JK flip-flop features include a toggle functionality, preventing illegal states, and a versatile behavior distinguishing it from an SR flip-flop.
8. **How is a JK flip-flop used to create a toggle circuit?**
   Connecting J and K inputs allows a JK flip-flop to function as a toggle circuit, changing state on each clock edge.
9. **In what applications is a JK flip-flop preferred over other types of flip-flops?**
   JK flip-flops are preferred in applications requiring toggling behavior, memory storage, and sequential logic due to their versatility and prevention of illegal states.
10. **In what applications is a D flip-flop typically used for data storage and transfer?**
    D flip-flops are used in applications like registers, memory units, and sequential logic circuits for reliable data storage and transfer.

Scanned with OKEN Scanner